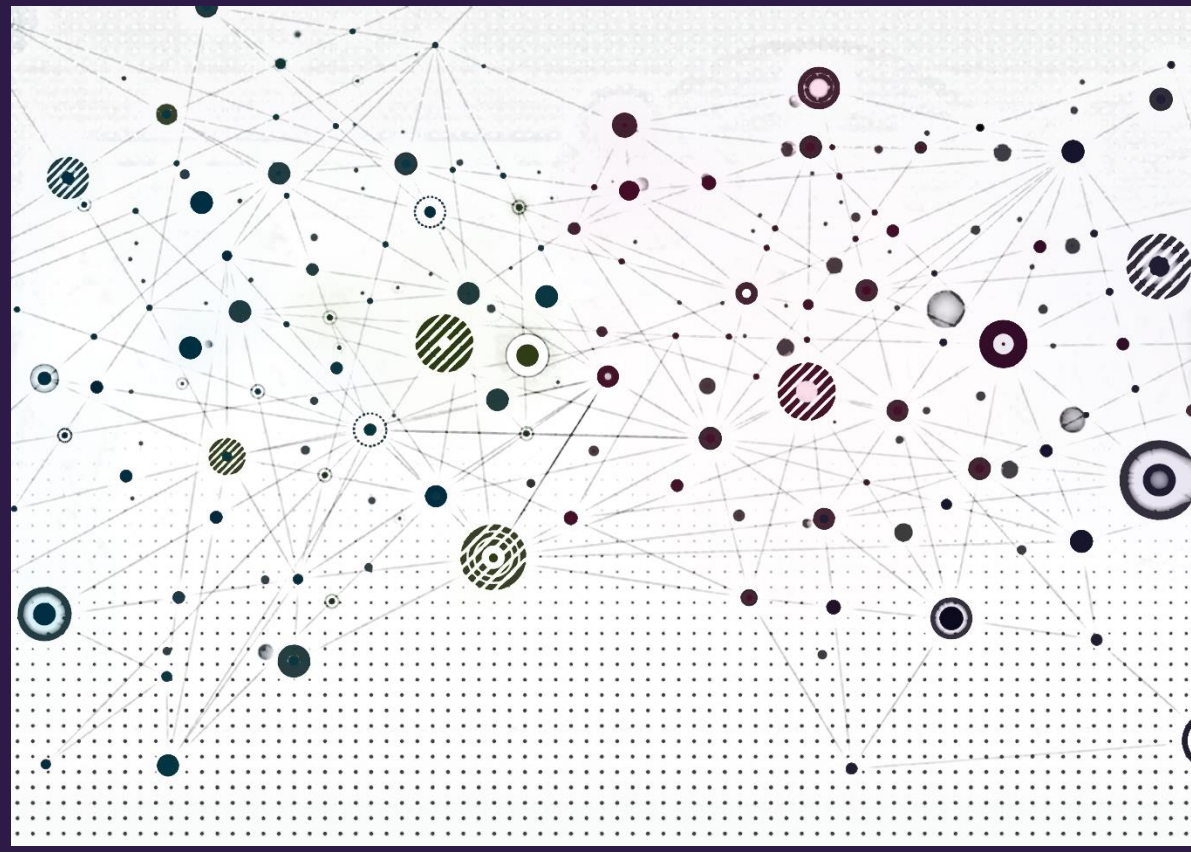
School of COMPUTING
Clemson University

Introduction to Graph Neural Networks



Overview

- Recap
- Deep learning meets graphs
- Graph convolutional networks

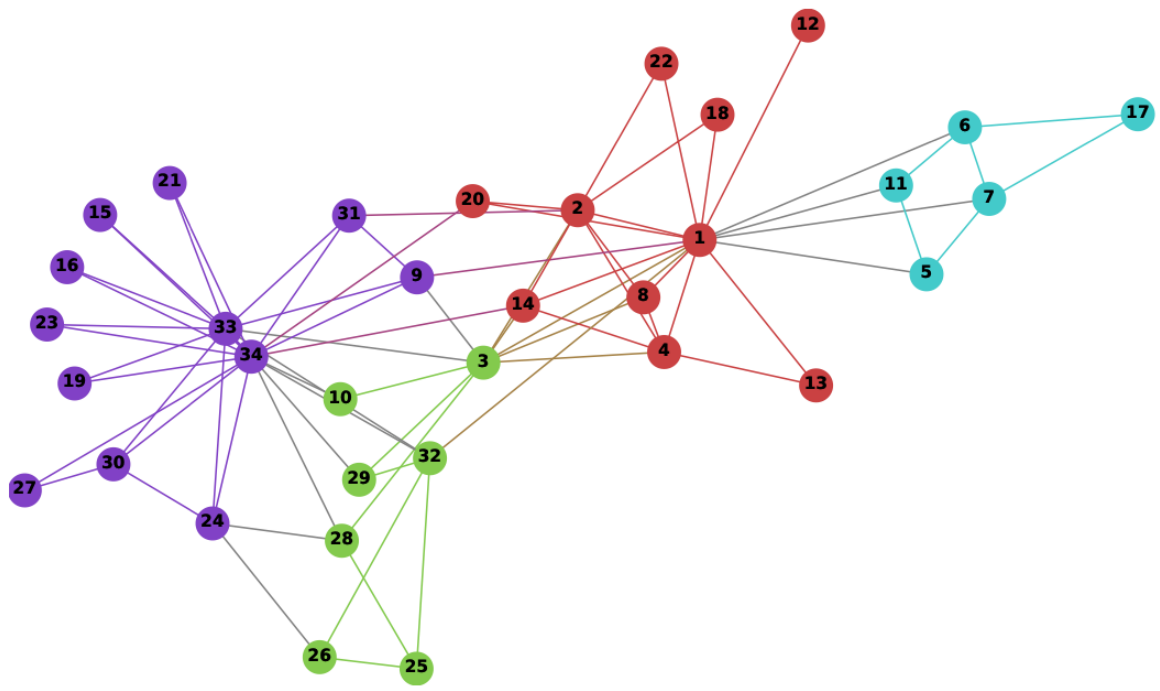
School of COMPUTING
Clemson University

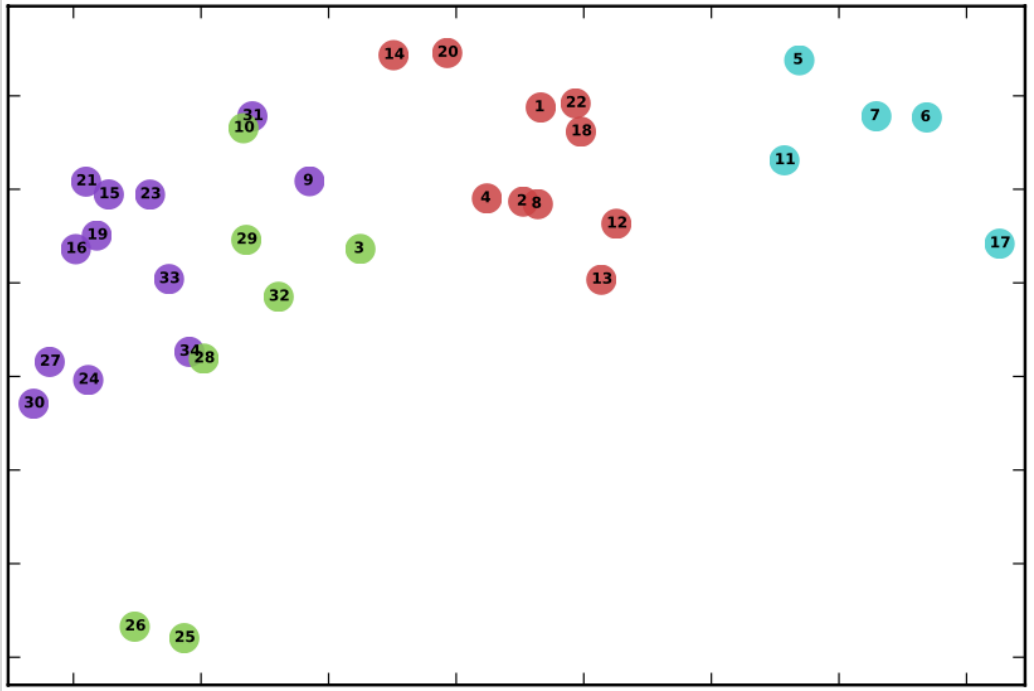
Recap

Node embeddings

Goal: Map nodes to $\mathbb{R}^d$ such that *similar* nodes are *close* to each other (e.g., have high cosine similarity)

Task: How to learn the mapping function?





Node embeddings

- Define an *encoder*, $ENC(v) = \mathbf{z}_v$, that maps v to a vector, $\mathbf{z}_v \in \mathbb{R}^d$
- Define a decoder $DEC(u, v) \approx \mathbf{S}[u, v]$ where $\mathbf{S}$ is a similarity function
- Define the encoder, decoder, and similarity and train

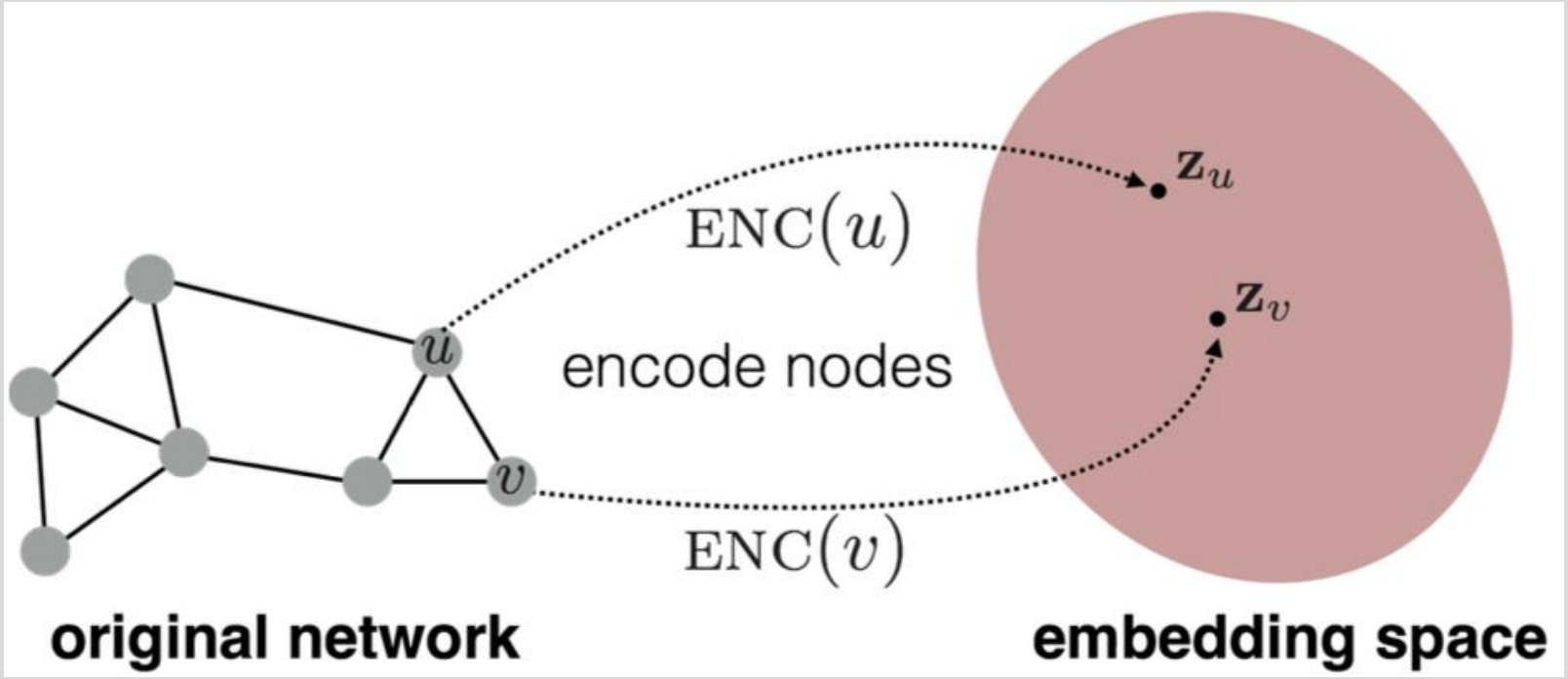
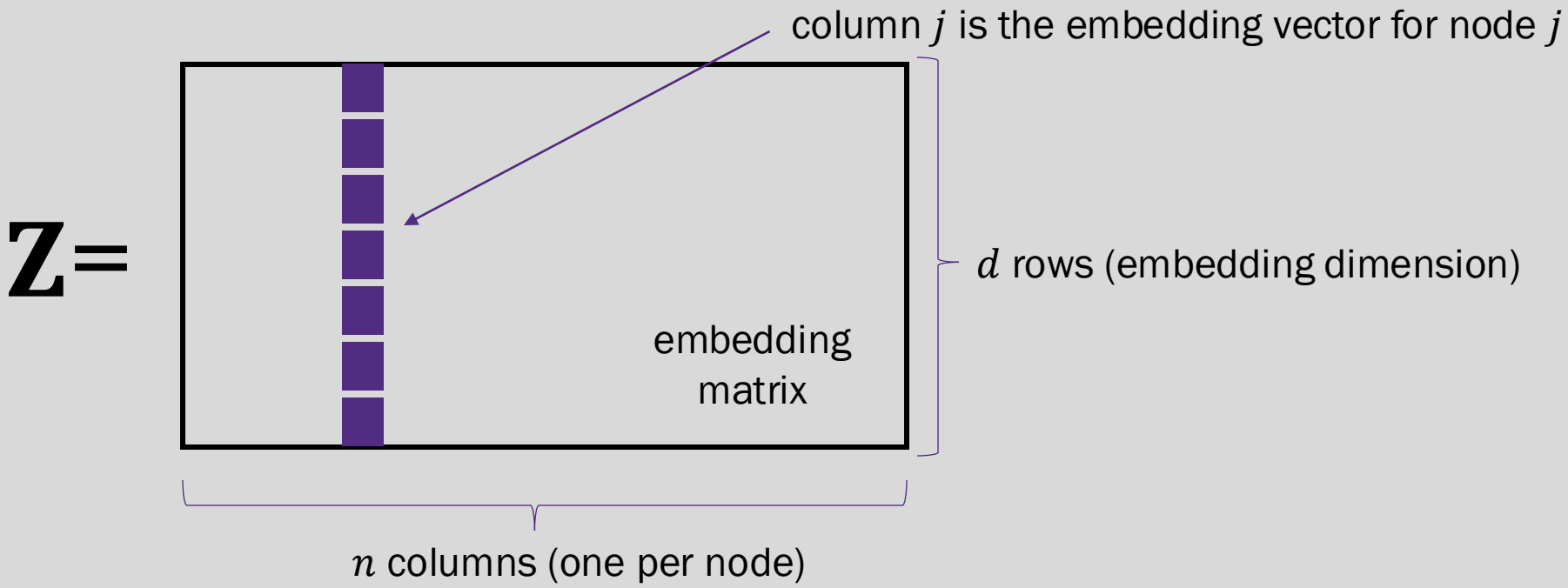


Image credit: Hamilton 2020

Shallow embeddings

- Simplest approach, the encoder is just a lookup



Shallow embedding limitations

- Transductive (not inductive) method – cannot generate embeddings for nodes / graphs not seen in training
- No parameter sharing, $O(|V|d)$ parameters required
- Do not incorporate node features

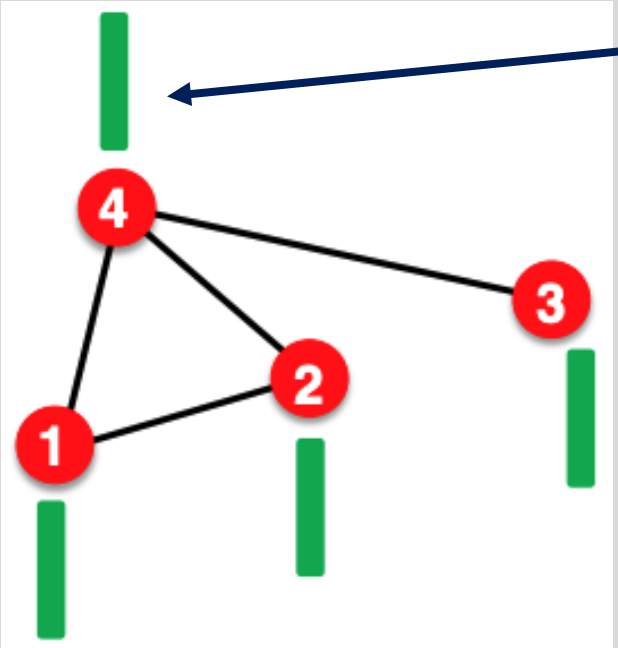


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Node feature vector
(e.g. protein properties in a protein-protein interaction graph)

Matrix factorization / random walk embeddings do not incorporate such node features

Solution: Graph neural networks

Deep learning meets graphs

Deep graph encoders (going beyond shallow embeddings)

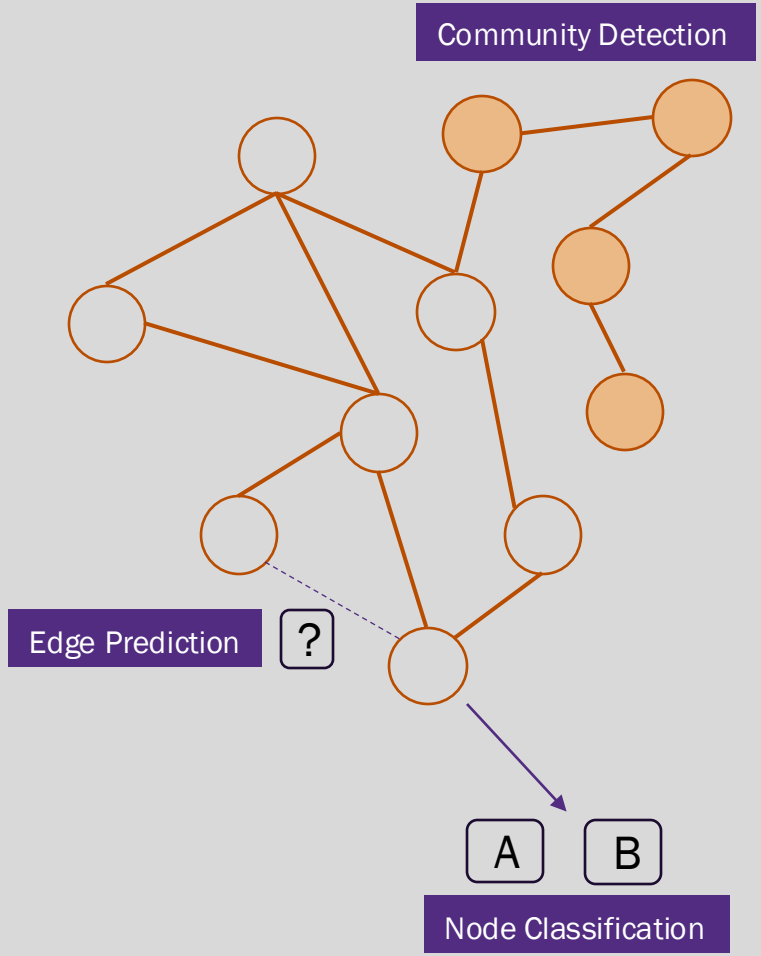
- Today, we begin discussing graph neural networks
  - Field of deep learning approaches for graph data
  - A defining characteristic is the encoder

$$ENC(v) \equiv$$ multiple iterations (layers) of non-linear transformations based on graph structure

- We may also use a decoder (though not always), $DEC(u, v) \approx \mathbf{S}[u, v]$ where $\mathbf{S}$ can be any node similarity function

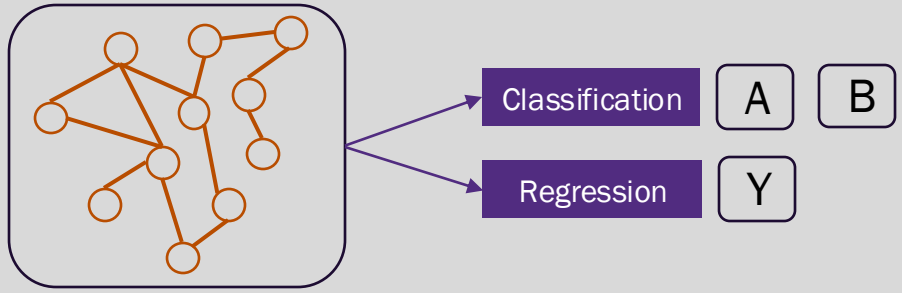
Intra-graph machine learning tasks

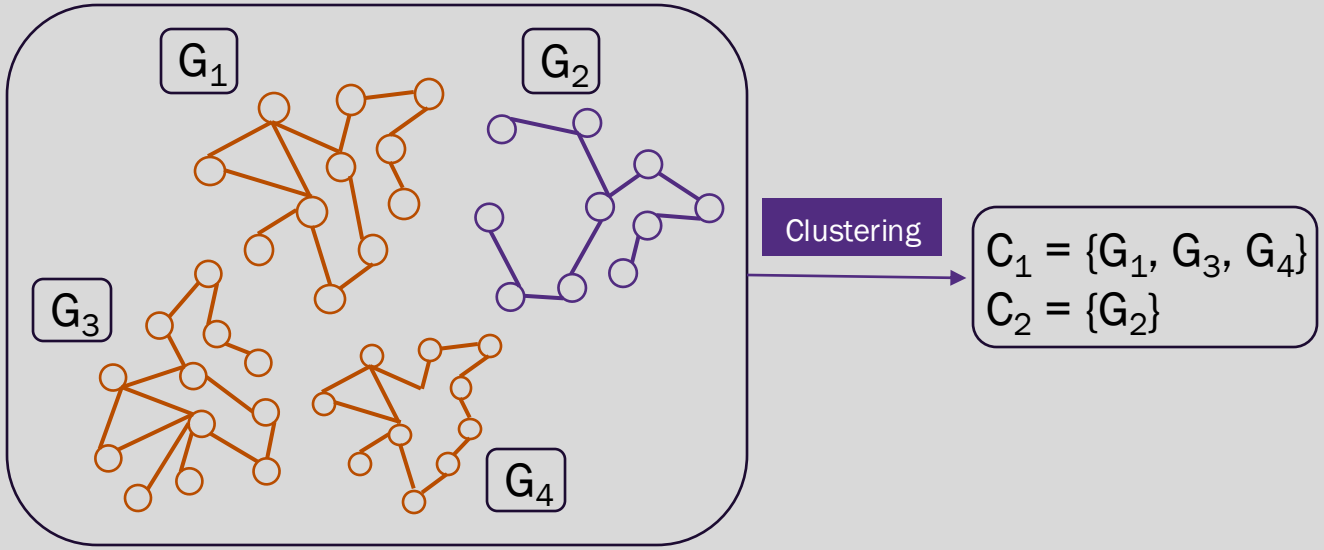
- Node classification – given information about the node (may include neighbors, relationships, attributes), infer class membership (usually an attribute)
- Edge prediction – given information about two nodes, determine if a given relation exists between them
- Community detection (node clustering) – detection of “meaningful” node groups



Whole graph and inter-graph machine learning tasks

- Whole graph classification and regression – given one entire graph, infer class membership or estimate (regress) a related numerical property
- Inter graph tasks – given multiple graphs estimate the similarity between graphs and/or identify clusters





Borrowing concepts from deep learning

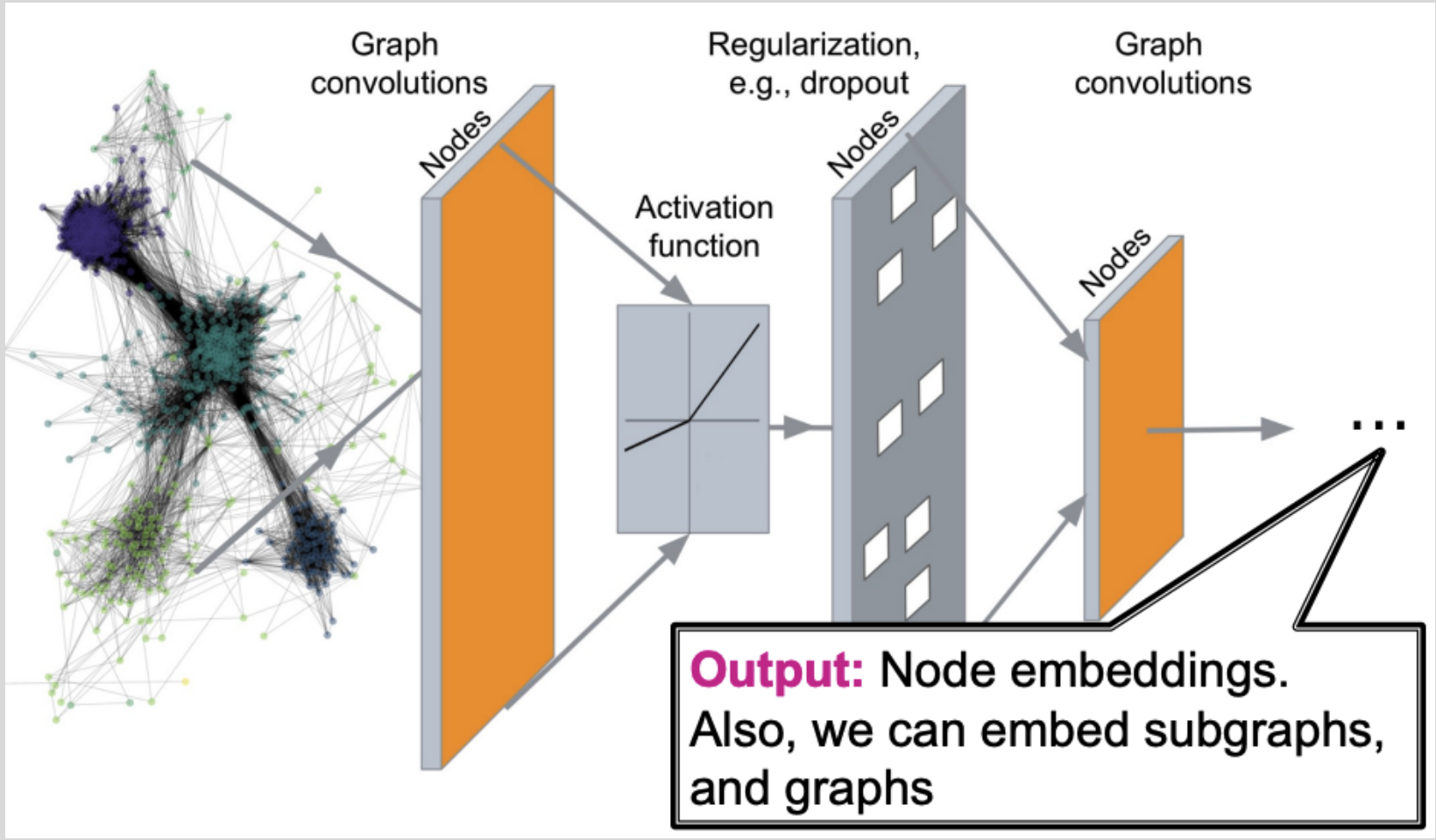
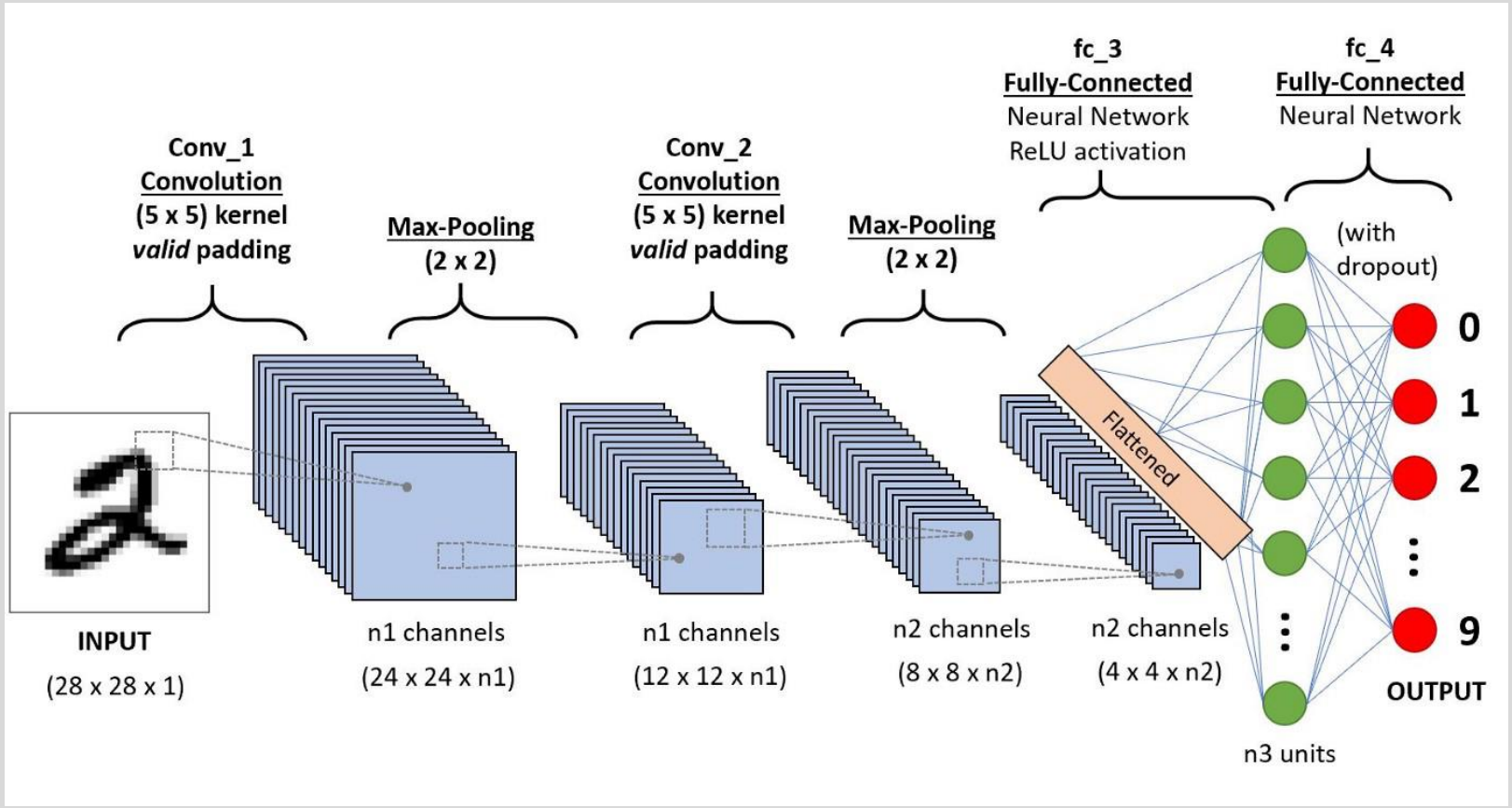
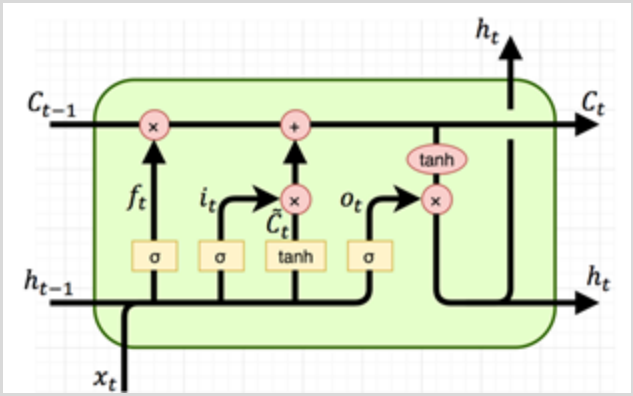


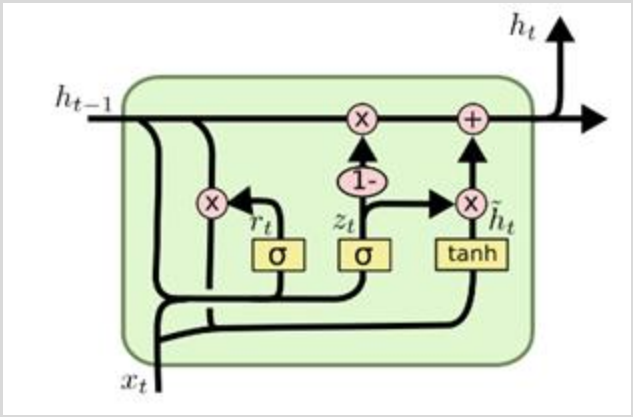
Image credit: Jure Leskovec, http://cs224w.stanford.edu

Why can't we just use existing methods?

Standard ML methods and deep learning assume fixed-sized and ordered inputs

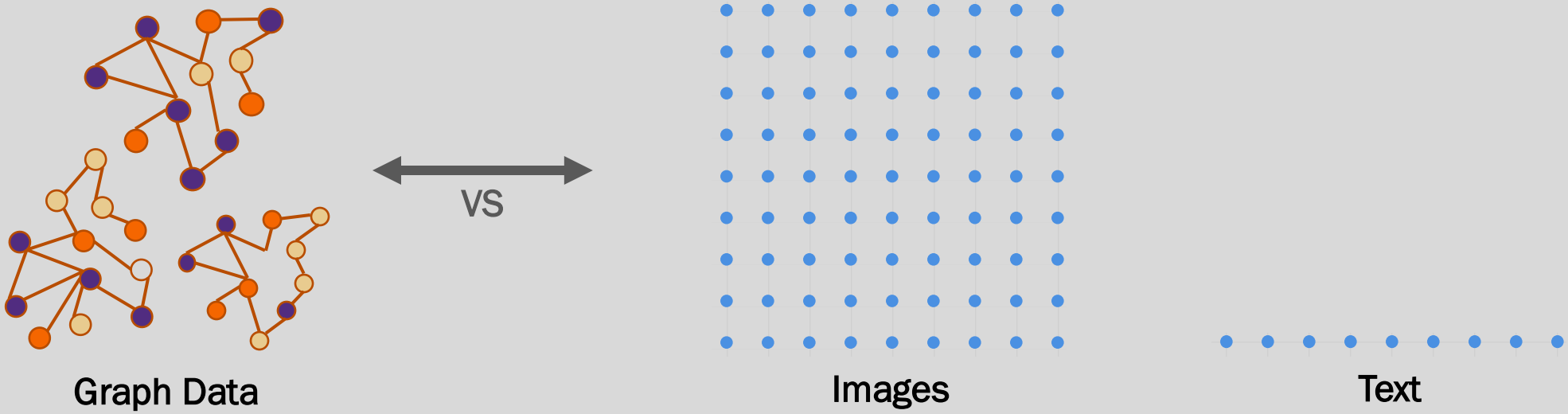






Modern ML toolbox assumes regularity

- Graphs contain arbitrary node and edge counts
- Graph nodes do not have an objective ordering
- Different node types may have different features



Review of deep learning basics

- Select a neural network model, $f_\Theta(\boldsymbol{x})$, parameterized on Θ that estimates the unknown relationship, f, between the input, $\boldsymbol{x}$, and an output, y.
  - f_Θ can be any differentiable network (linear layer, multilayer perceptron, CNN, GNN)
- Select a loss function $\mathcal{L}$ to minimize

$$\min_{\Theta} \mathcal{L}\left(y, f_\Theta(\boldsymbol{x})\right)$$

- Sample minibatches of input and output samples $\boldsymbol{x}, \boldsymbol{y}$
- **Forward propagation:** Compute $\mathcal{L}$ form minibatch samples
- **Back-propagation:** Obtain $\nabla_\Theta \mathcal{L}$ using chain rule of differentiation
- Apply **stochastic gradient descent (SGD)** to update model parameters, Θ, in the negative direction of $\nabla_\Theta \mathcal{L}$

How can we do this for graph networks?

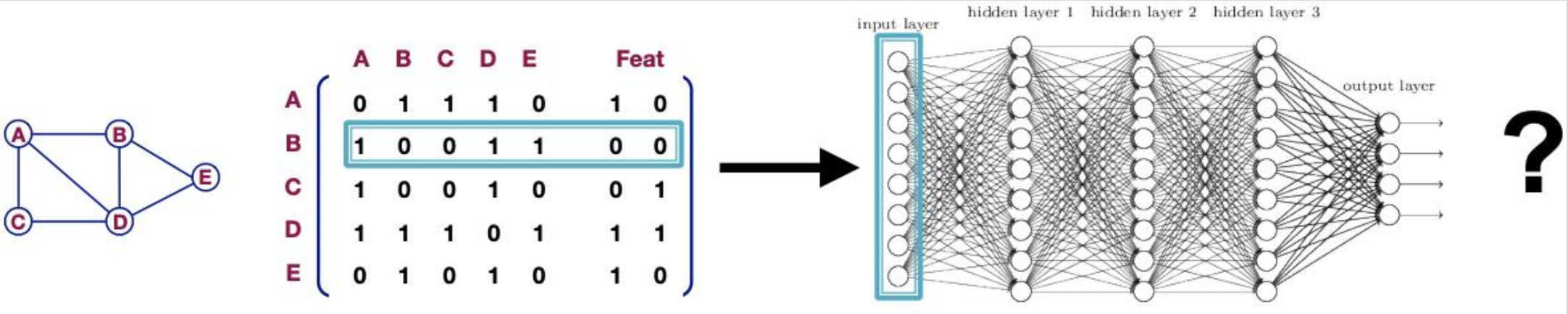
Graph deep learning paradigm

- Assume we have a graph $\boldsymbol{G}$
- V is the set of vertices in $\boldsymbol{G}$
- $\mathbf{A}$ is the adjacency matrix (assume unweighted, undirected for now)
- $\mathbf{X} \in \mathbb{R}^{|V| \times d_l}$ is a matrix of **node features (this is new!)**
  - One d_l-dimensional feature vector per node
  - Examples of node features
    - User profile data, user image
    - Gene expression, gene functional properties
  - What if my nodes don't have features?
    - One-hot embeddings - not recommended as this makes the model transductive
    - Constant vector of 1 – model is again inductive, features will be updated (more to come)

A naïve approach to graph deep learning

- Concatenate the adjacency matrix and features into a single feature matrix
- Provide as input to a feed-forward deep neural network

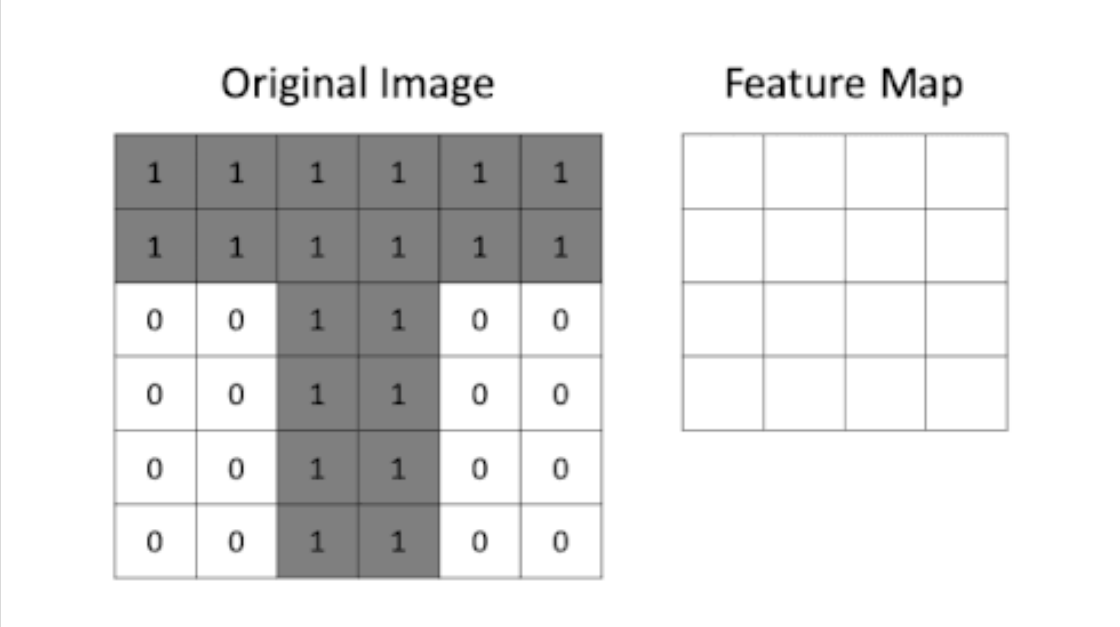
Image credit: Jure Leskovec, http://cs224w.stanford.edu

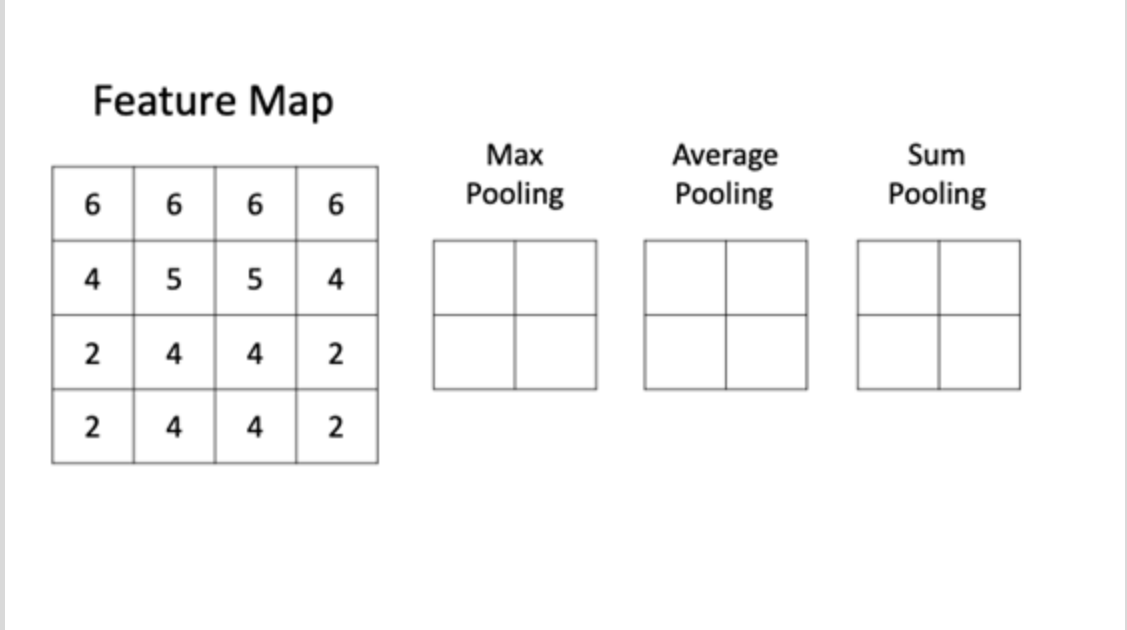


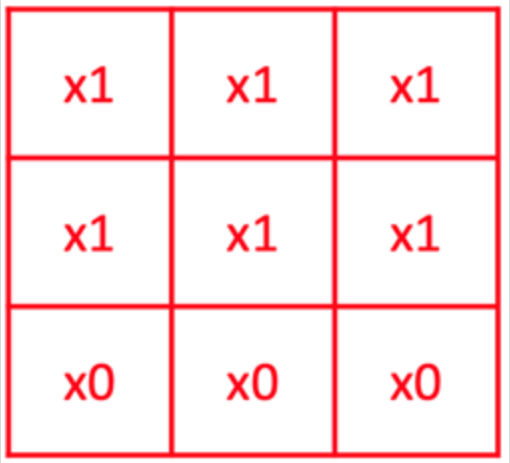
- Serious limitations
  - $O(|V| + d)$ parameters where d is node feature vector dimension
  - Model is transductive
  - Model is sensitive to arbitrary node ordering

What else can we do?

Can we apply convolutional network approaches?







Max pooling is common, but average and sum pooling are also used

Can we apply convolutional network approaches?

- Generalize convolution beyond lattices (images)
- Leverage node features

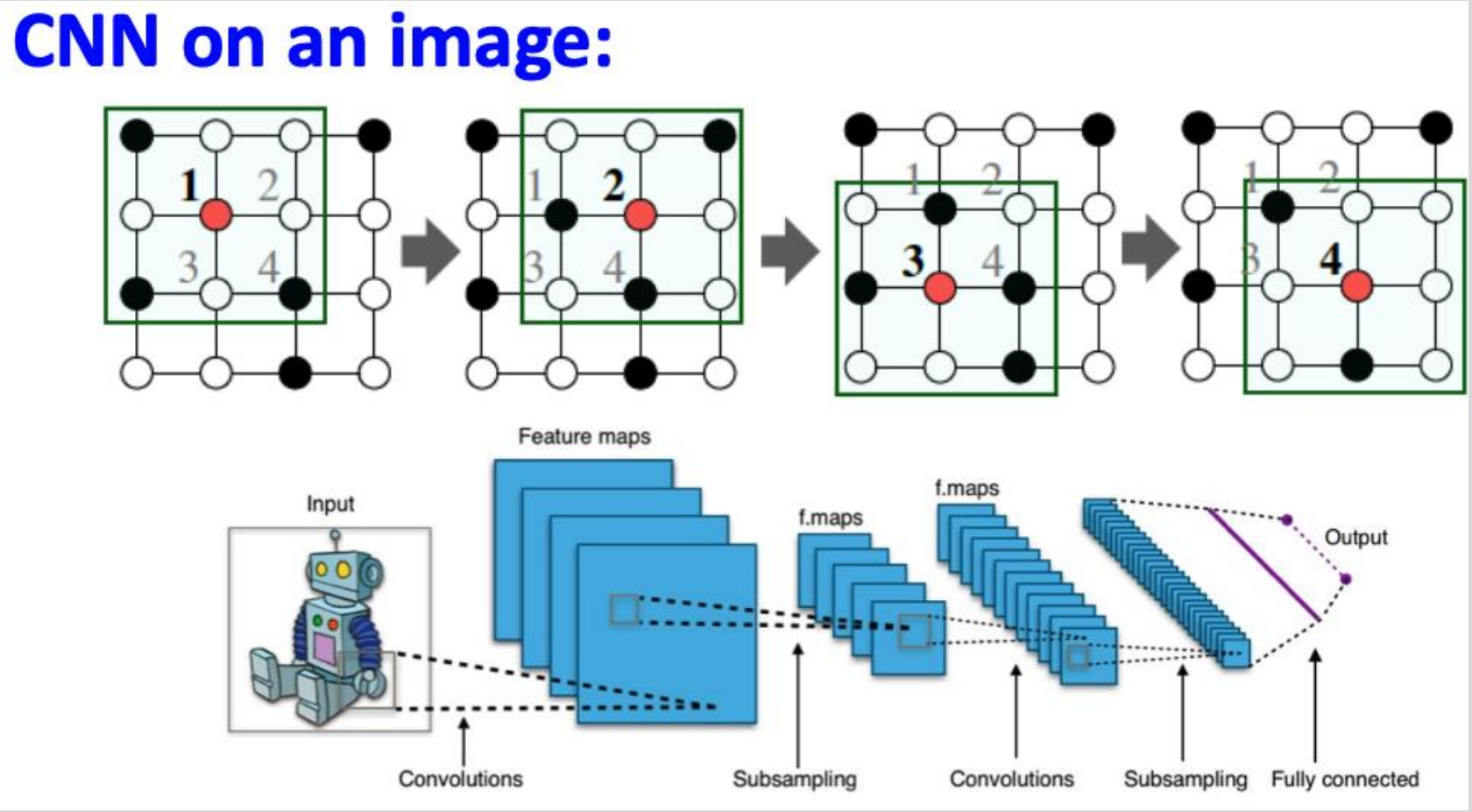


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Can we apply convolutional network approaches?

- Graphs are generally **NOT** regular lattices
- There is no fixed reference (locality) for a sliding window (i.e., the convolution filter)
- Graphs are **permutation invariant**

What does this mean?

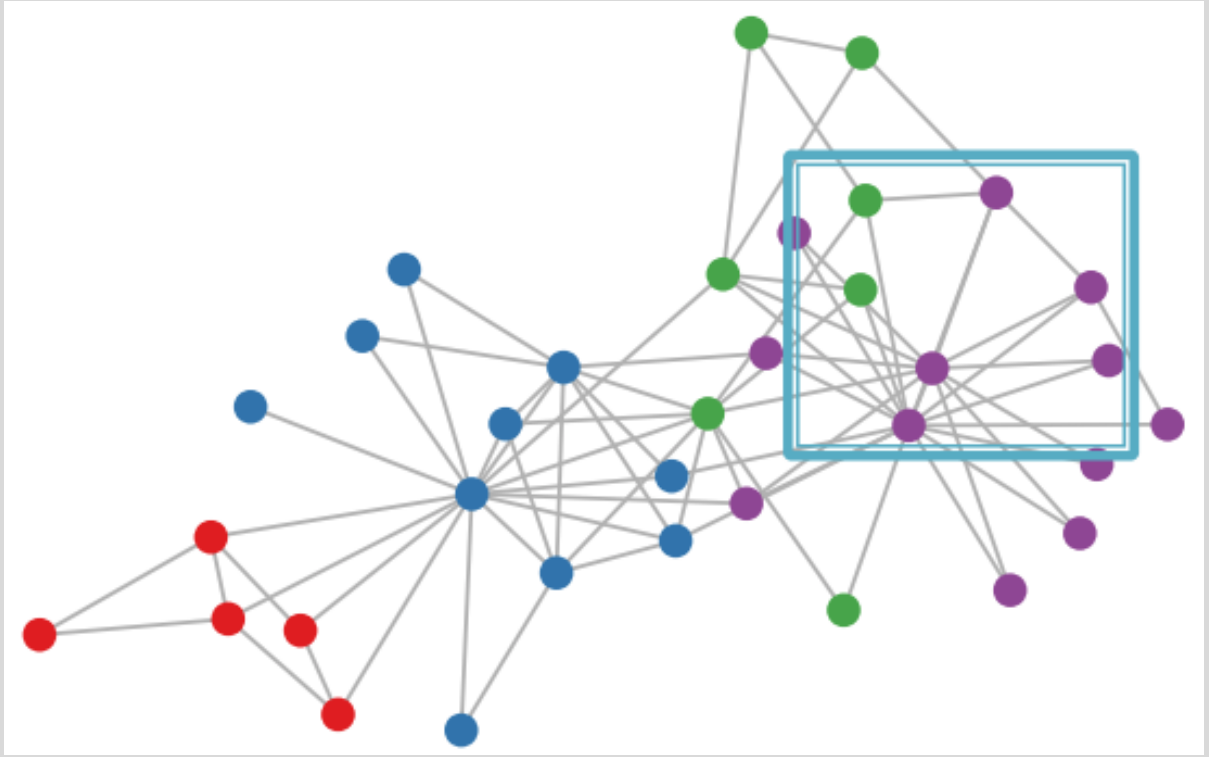
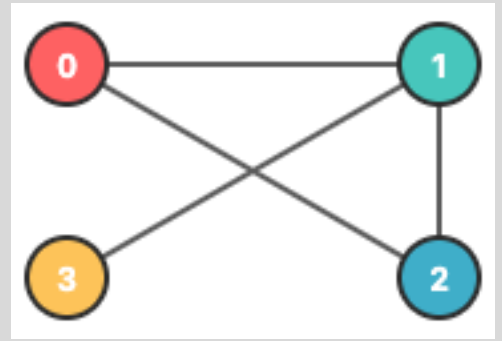
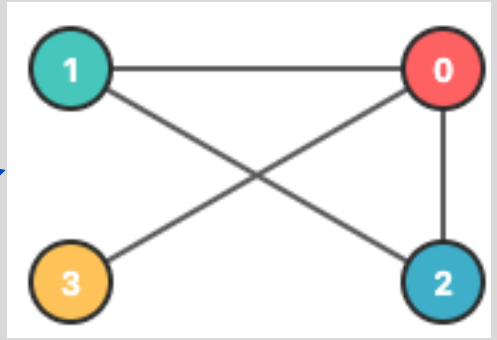


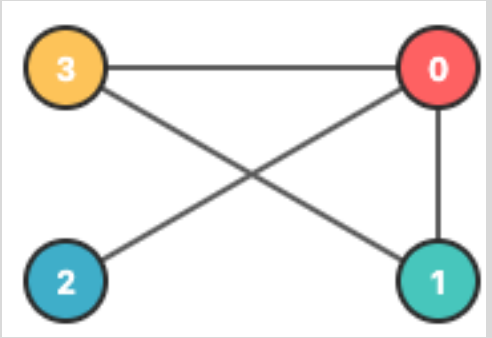
Image credit: Jure Leskovec, http://cs224w.stanford.edu

Permutation invariance

- Graphs do not have a canonical node ordering
  - We can arbitrarily assign different node orders without altering the graph structure
- Desiderata – a **graph** embedding function should produce the same result (**graph embedding**) for any node ordering



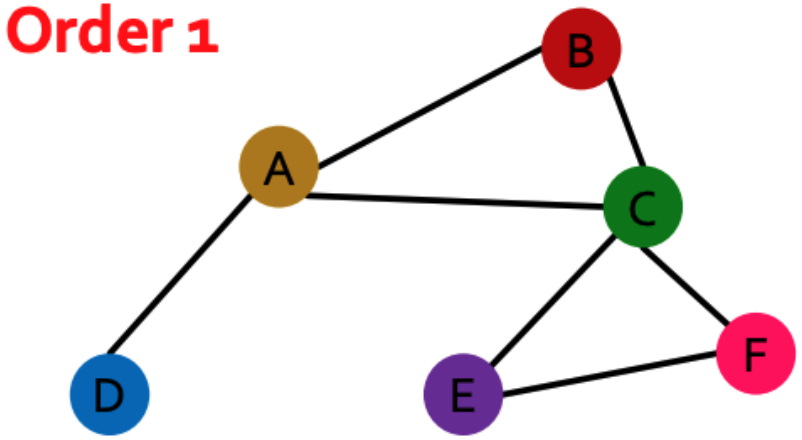


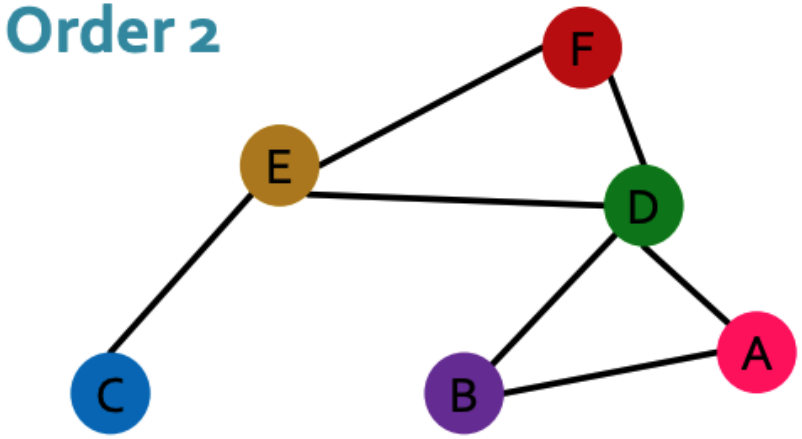


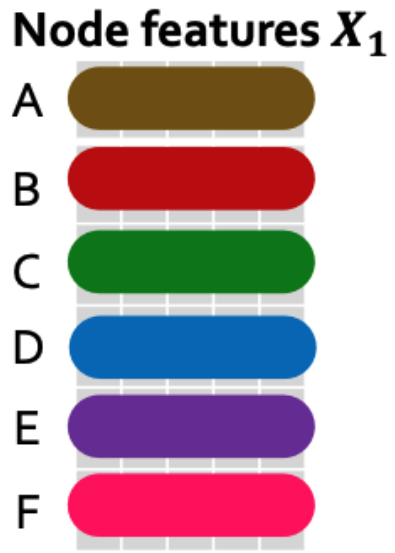
Structurally equivalent

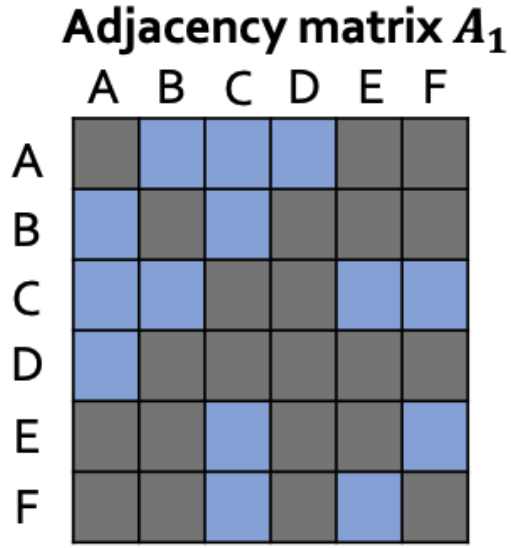
https://ml4g.masinolab.com/topics/gnns/gnn-permutation-demo.html

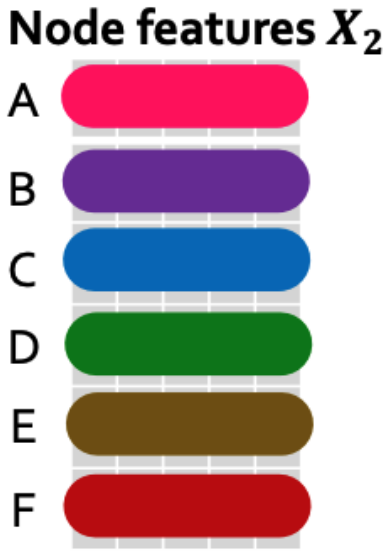
Permutation invariance

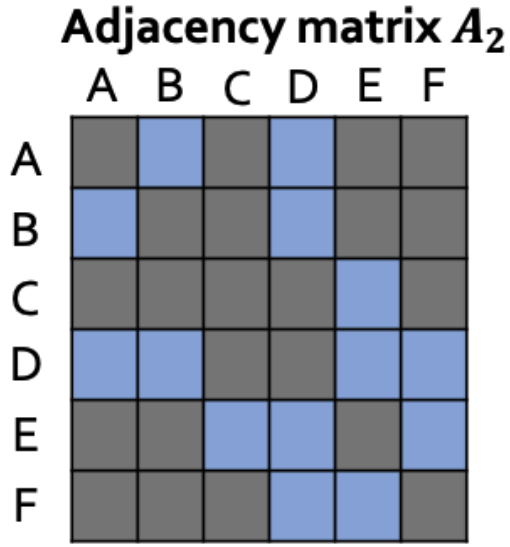












Ordering changes node position index in A

Learned graph representation should be the same for any ordering

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Permutation invariance formal definition

- Consider a function $f(\mathbf{A}, \boldsymbol{X}) \longrightarrow \mathbb{R}^{d_{out}}$ where $\mathbf{A}$ is a graph adjacency matrix and $\boldsymbol{X}$ is a node feature matrix
- If $f(\mathbf{A}_i, \boldsymbol{X}_i) = f\left(\mathbf{A}_j, \boldsymbol{X}_j\right)$ for any node ordering i and j, then f is a **permutation invariant function**
  - There $|V|!$ different orderings for a graph with $|V|$ nodes. Why?
- Let's refine this definition for graph node embeddings
- A function $f(\mathbf{A}, \boldsymbol{X})$: $\mathbb{R}^{|V|\times|V|} \times \mathbb{R}^{|V|\times d_{in}} \longrightarrow \mathbb{R}^{d_{out}}$ is permutation invariant if

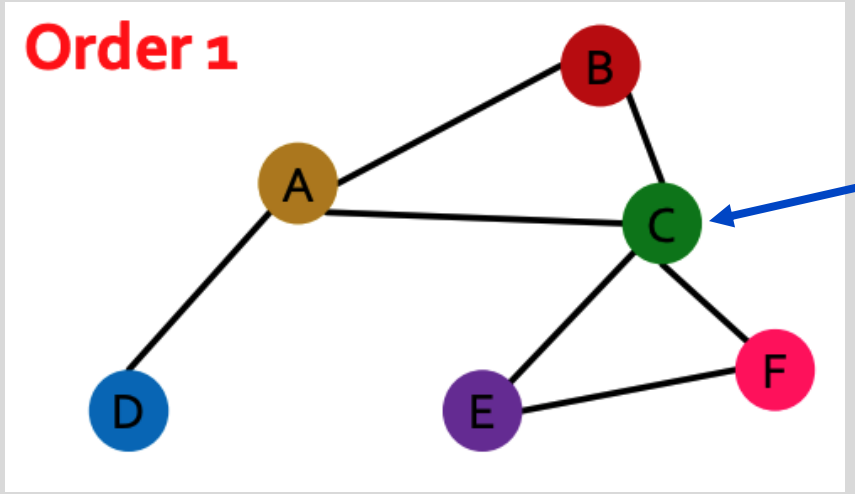
$$f(\mathbf{A}, \boldsymbol{X}) = f\left(\mathbf{P}\mathbf{A}\boldsymbol{P}^{\boldsymbol{T}}, \boldsymbol{P}\boldsymbol{X}\right)$$

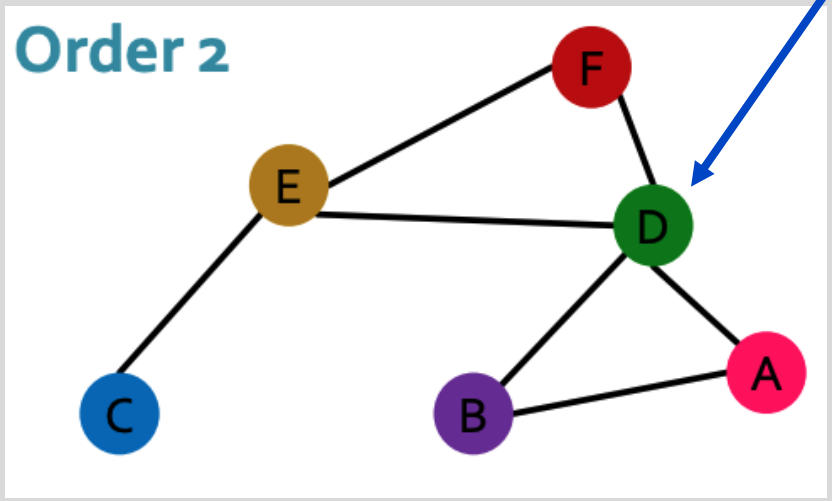
where $\mathbf{P}$ is a permutation matrix

  - $\mathbf{P}$ is a matrix with exactly one 1 in each row, column (zero for other values)
  - Left multiplication by $\mathbf{P}$ swaps rows
  - Right multiplication by $\mathbf{P}$ swaps columns

https://ml4g.masinolab.com/topics/gnns/gnn-permutation-demo.html

Permutation invariance – node level





For any target node

$$f(\mathbf{A}, \boldsymbol{X}) = f\left(\mathbf{P}\mathbf{A}\boldsymbol{P}^{\boldsymbol{T}}, \boldsymbol{P}\boldsymbol{X}\right)$$

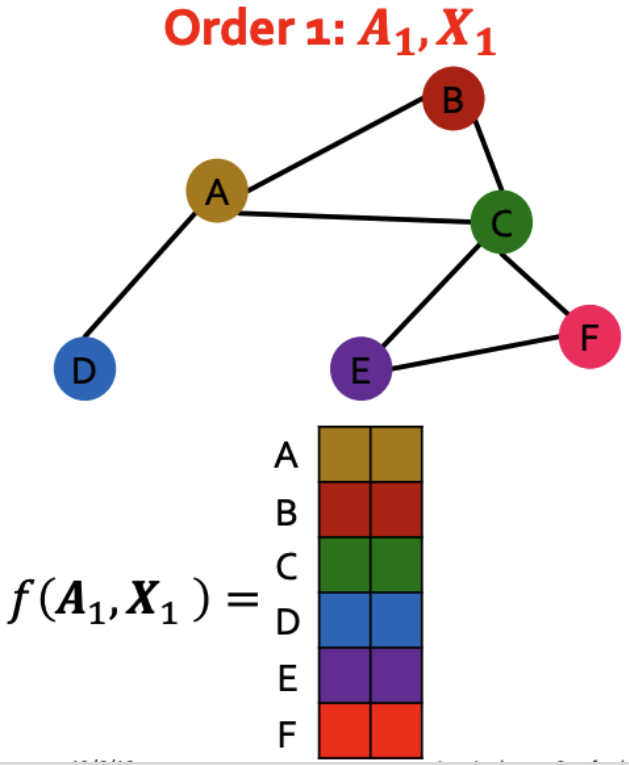
Same node embedding

Image credit: Jure Leskovec, http://cs224w.stanford.edu

What about **graph** "permutation invariance"?
Let's talk about ***equivariance.***

Permutation equivariance

- As with shallow embeddings, with GNN we will learn a function, $f: ENC(v)$, that maps nodes of $\boldsymbol{G}$ to a matrix $\mathbb{R}^{|V| \times d_{out}}$
  - Each node is mapped to a d_{out}-dimensional embedding



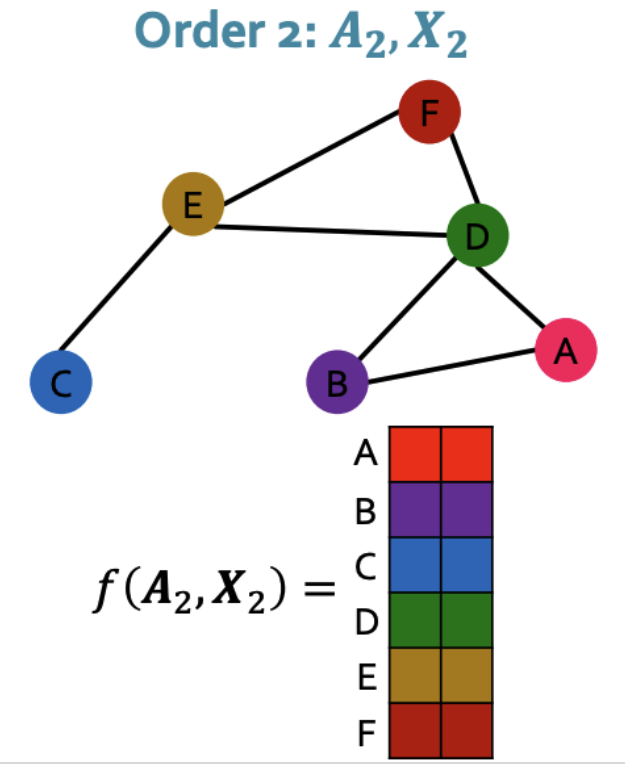
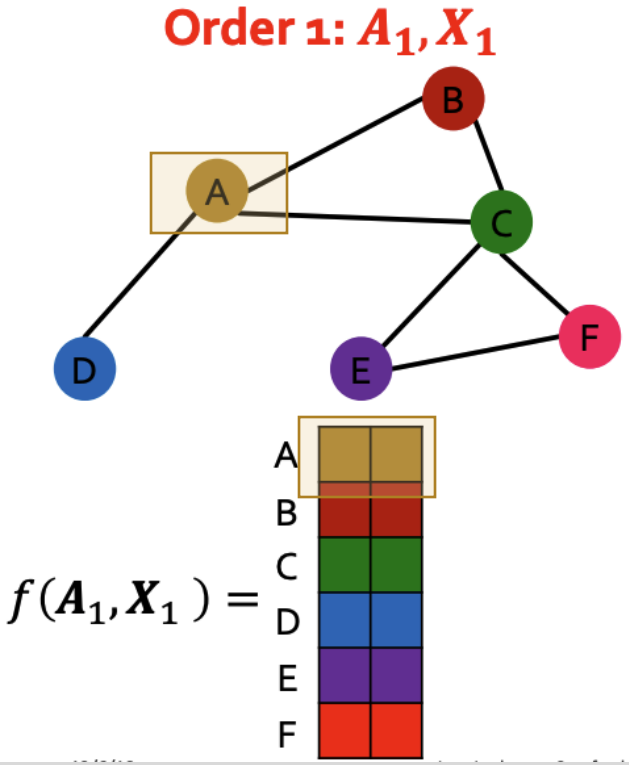
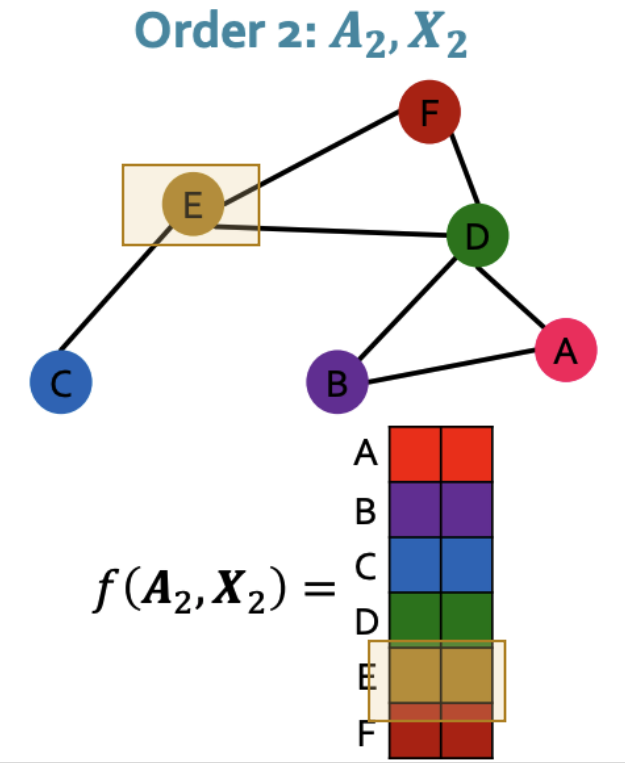


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Permutation equivariance

- As with shallow embeddings, with GNN we will learn a function, $f: ENC(v)$, that maps nodes of $\boldsymbol{G}$ to a matrix $\mathbb{R}^{|V| \times d_{out}}$
  - Each node is mapped to a d_{out}-dimensional embedding



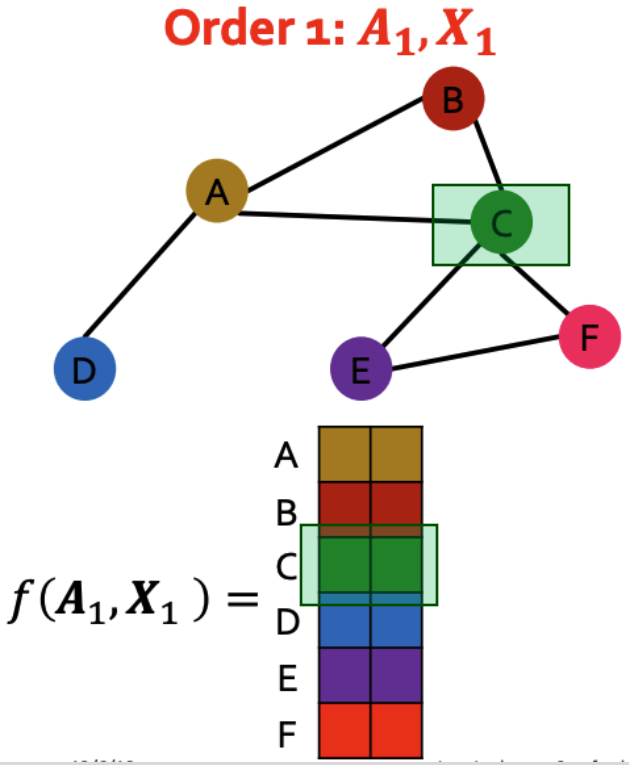


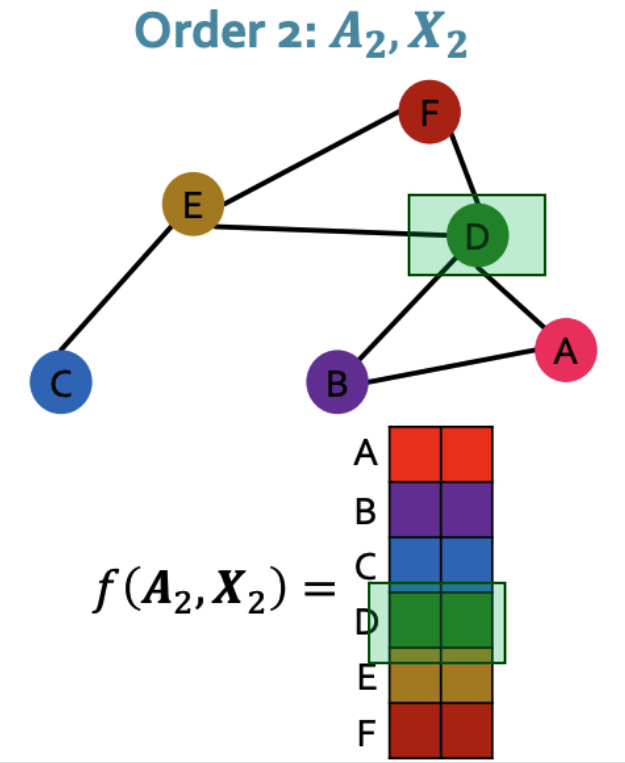
Node A (Order 1) is in the same position as node E (Order 2). It should have the same embedding (invariance)

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Permutation equivariance

- As with shallow embeddings, with GNN we will learn a function, $f: ENC(v)$, that maps nodes of $\boldsymbol{G}$ to a matrix $\mathbb{R}^{|V| \times d_{out}}$
  - Each node is mapped to a d_{out}-dimensional embedding



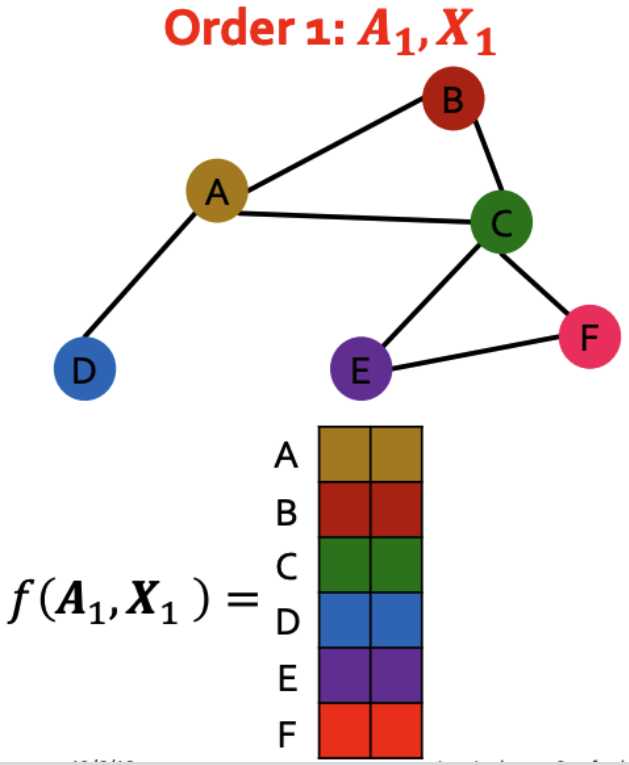


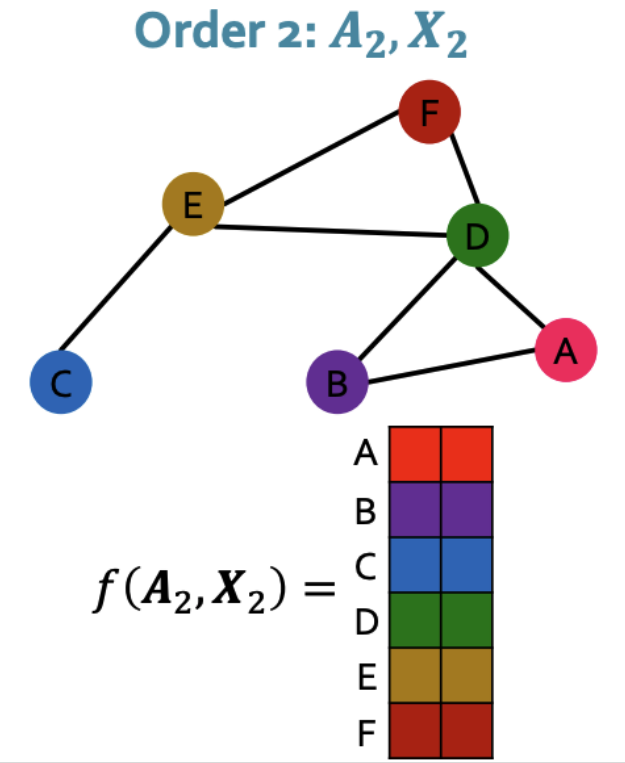
Node C (Order 1) is in the same position as node D (Order 2). It should have the same embedding (invariance)

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Permutation equivariance

- As with shallow embeddings, with GNN we will learn a function, $f: ENC(v)$, that maps nodes of $\boldsymbol{G}$ to a matrix $\mathbb{R}^{|V| \times d_{out}}$
  - Each node is mapped to a d_{out}-dimensional embedding





The matrix of all node embeddings should be the same except for row shuffling (permutation)

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Permutation equivariance formal definition

- Consider a function $f(\mathbf{A}, \boldsymbol{X}) \longrightarrow \mathbb{R}^{|V| \times d_{out}}$ where $\mathbf{A}$ is a graph adjacency matrix and $\boldsymbol{X}$ is a node feature matrix
- If the embedding of a node (it's row in the output matrix) in the same graph position is unchanged for any node ordering, f is permutation equivariant
- Formally, a function $f(\mathbf{A}, \boldsymbol{X})$: $\mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times d_{in}} \longrightarrow \mathbb{R}^{d_{out}}$ is permutation equivariant if

$$\mathbf{P}f(\mathbf{A}, \boldsymbol{X}) = f\left(\mathbf{P}\mathbf{A}\boldsymbol{P}^{\boldsymbol{T}}, \boldsymbol{P}\boldsymbol{X}\right)$$

where $\mathbf{P}$ is a permutation matrix

f outputs a node embedding matrix which is permuted in the same manner as $\mathbf{A}$

https://ml4g.masinolab.com/topics/gnns/gnn-permutation-demo.html

Permutation invariance and equivariance

- Permutation invariant – permute the input, the output is unchanged

$$f(\mathbf{A}, \boldsymbol{X}) = f\left(\mathbf{P}\mathbf{A}\boldsymbol{P}^{\boldsymbol{T}}, \boldsymbol{P}\boldsymbol{X}\right)$$

- Permutation equivariance - permute the input, the output is permuted in the same way

$$\mathbf{P}f(\mathbf{A}, \boldsymbol{X}) = f\left(\mathbf{P}\mathbf{A}\boldsymbol{P}^{\boldsymbol{T}}, \boldsymbol{P}\boldsymbol{X}\right)$$

GNNs chain permutation invariant and equivariant operations

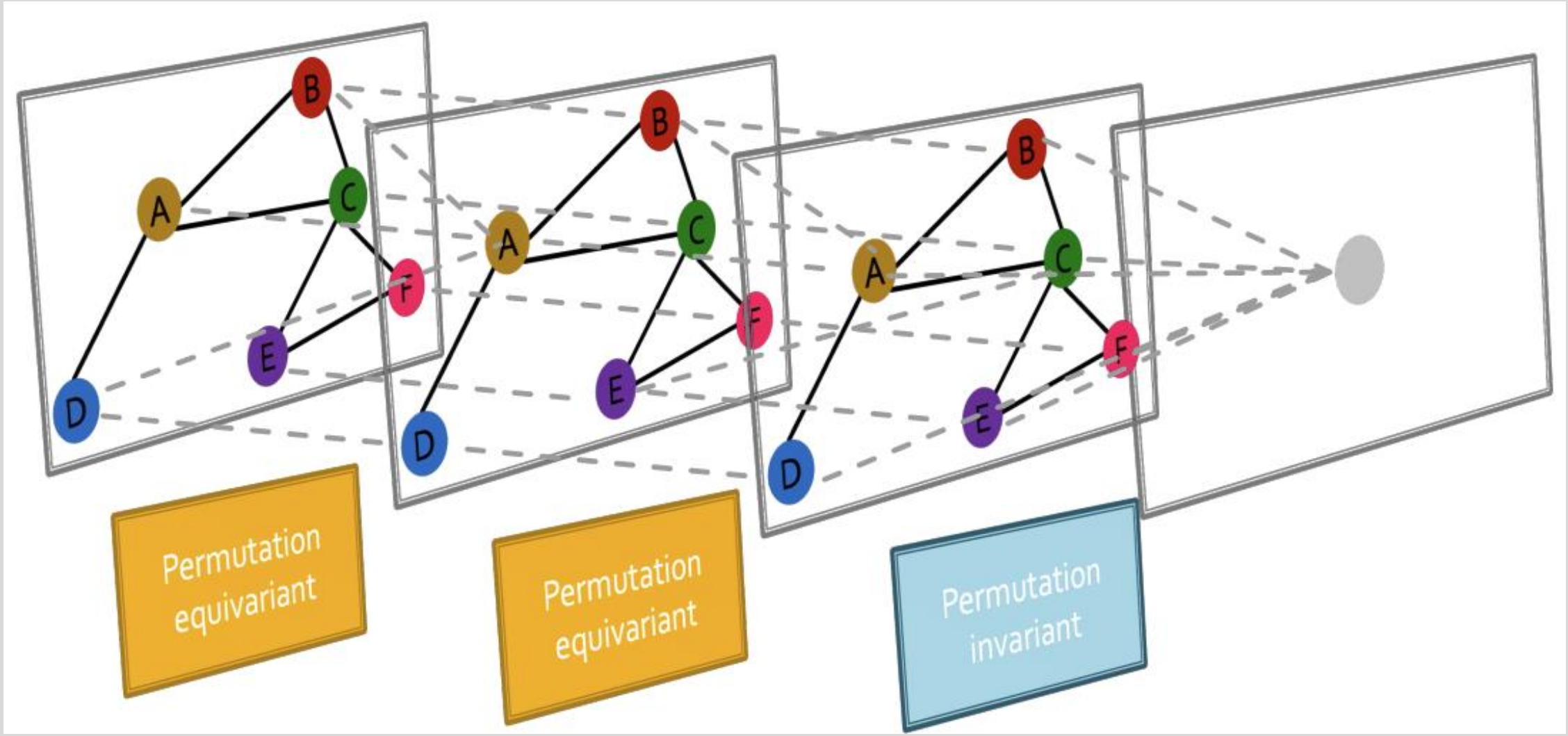


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Standard neural networks are permutation sensitive

Are common neural network architectures (linear layers, MLPs, CNNs, RNNs) permutation invariant / equivariant? **NO!!!**

Changing the input order changes the output. Why?

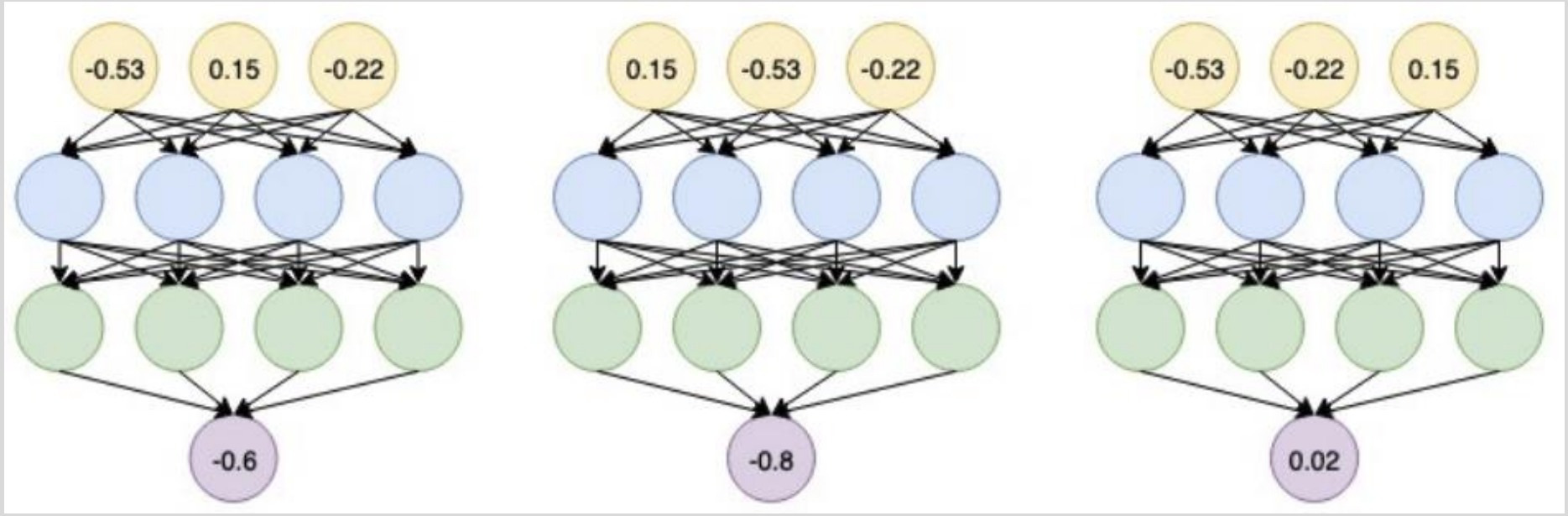
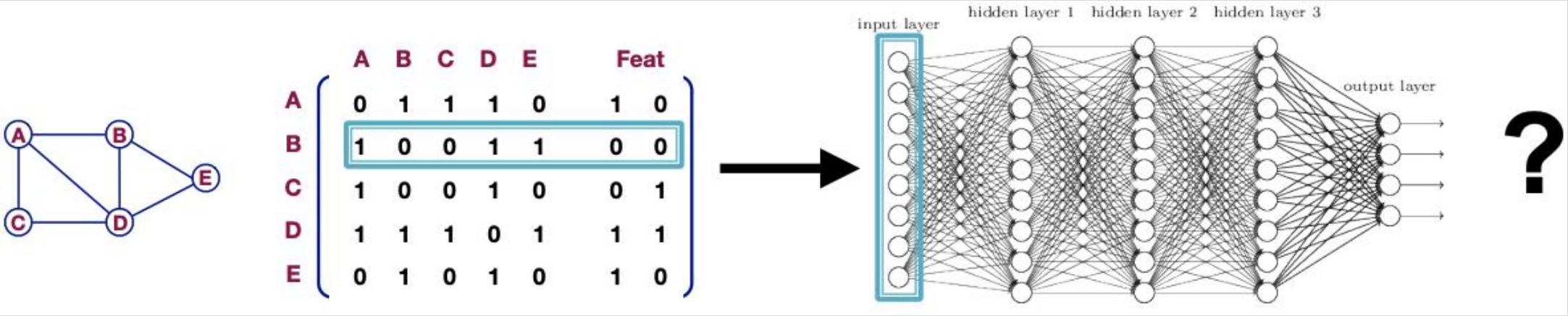


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Standard neural networks are permutation sensitive

Are common neural network architectures (linear layers, MLPs, CNNs, RNNs) permutation invariant / equivariant? **NO!!!**

Image credit: Jure Leskovec, http://cs224w.stanford.edu



This is why the naïve approach fails for graphs.

School of COMPUTING
Clemson University

Graph convolution networks

Neighbor node aggregation

- Generate node embeddings based on local network neighborhood

Input graph

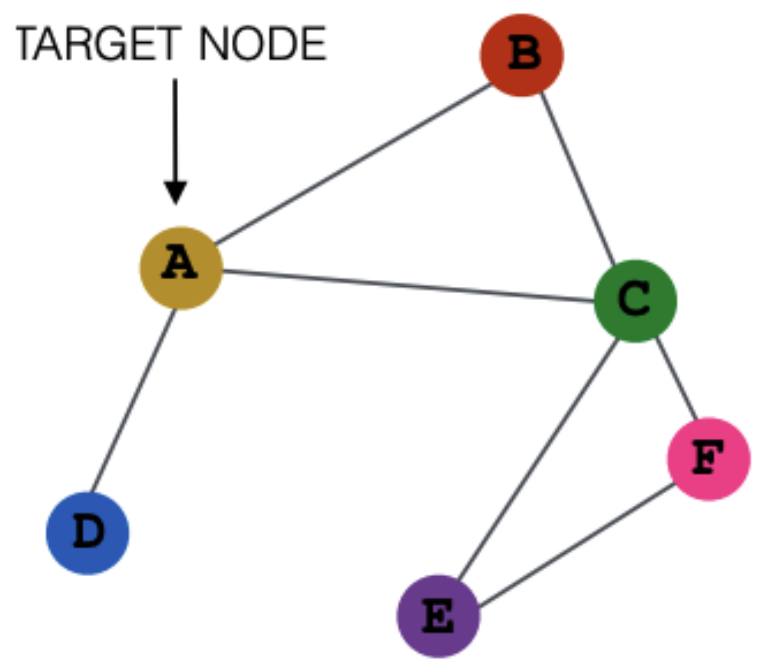
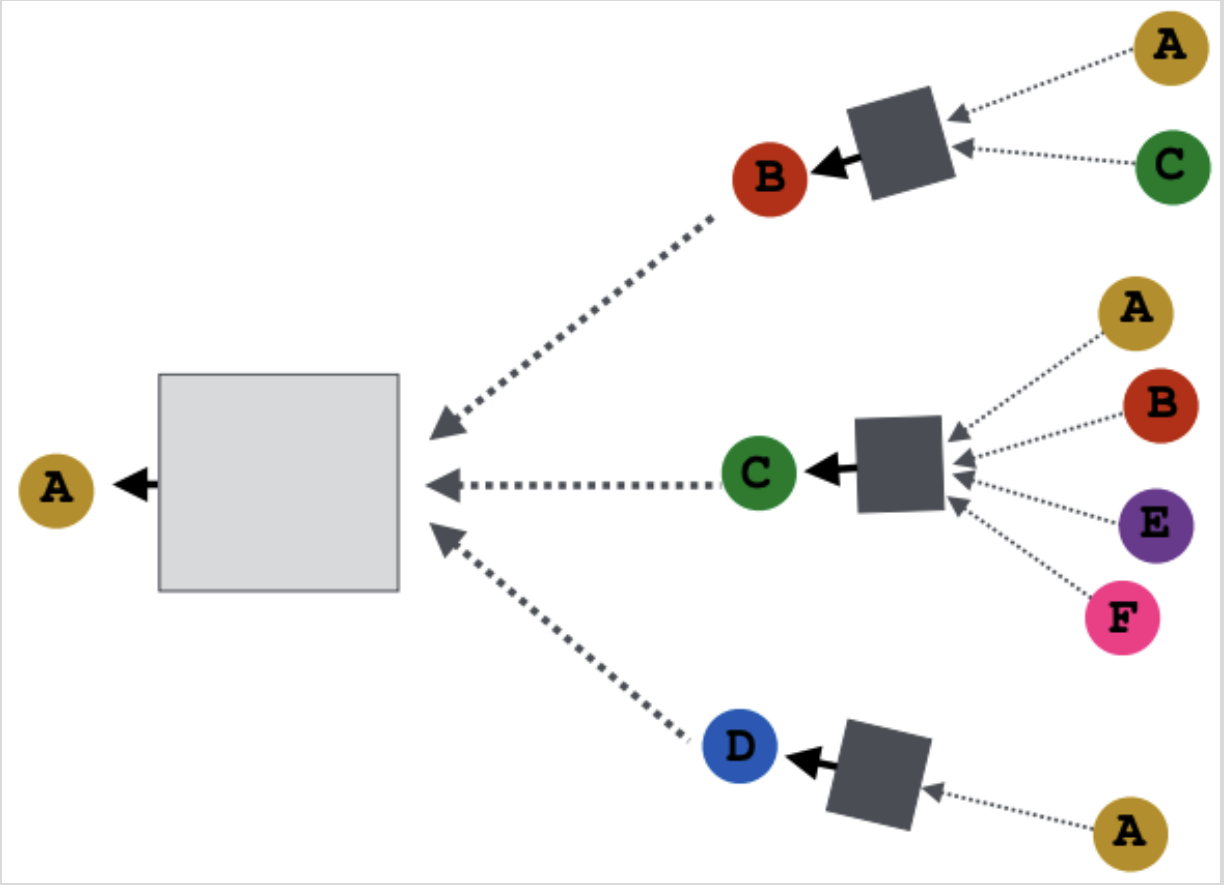


Image credit: Jure Leskovec, http://cs224w.stanford.edu



Neighbor node aggregation

- Generate node embeddings based on local network neighborhood

Input graph

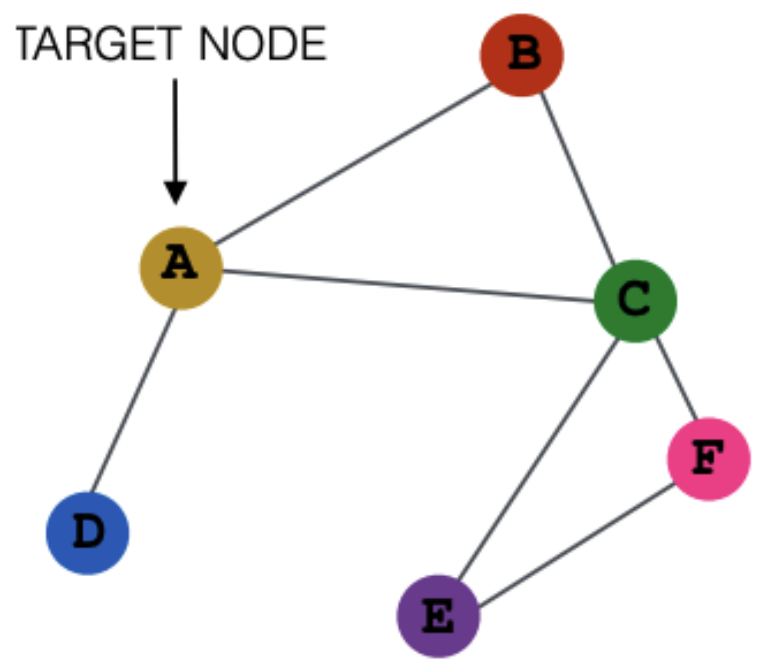
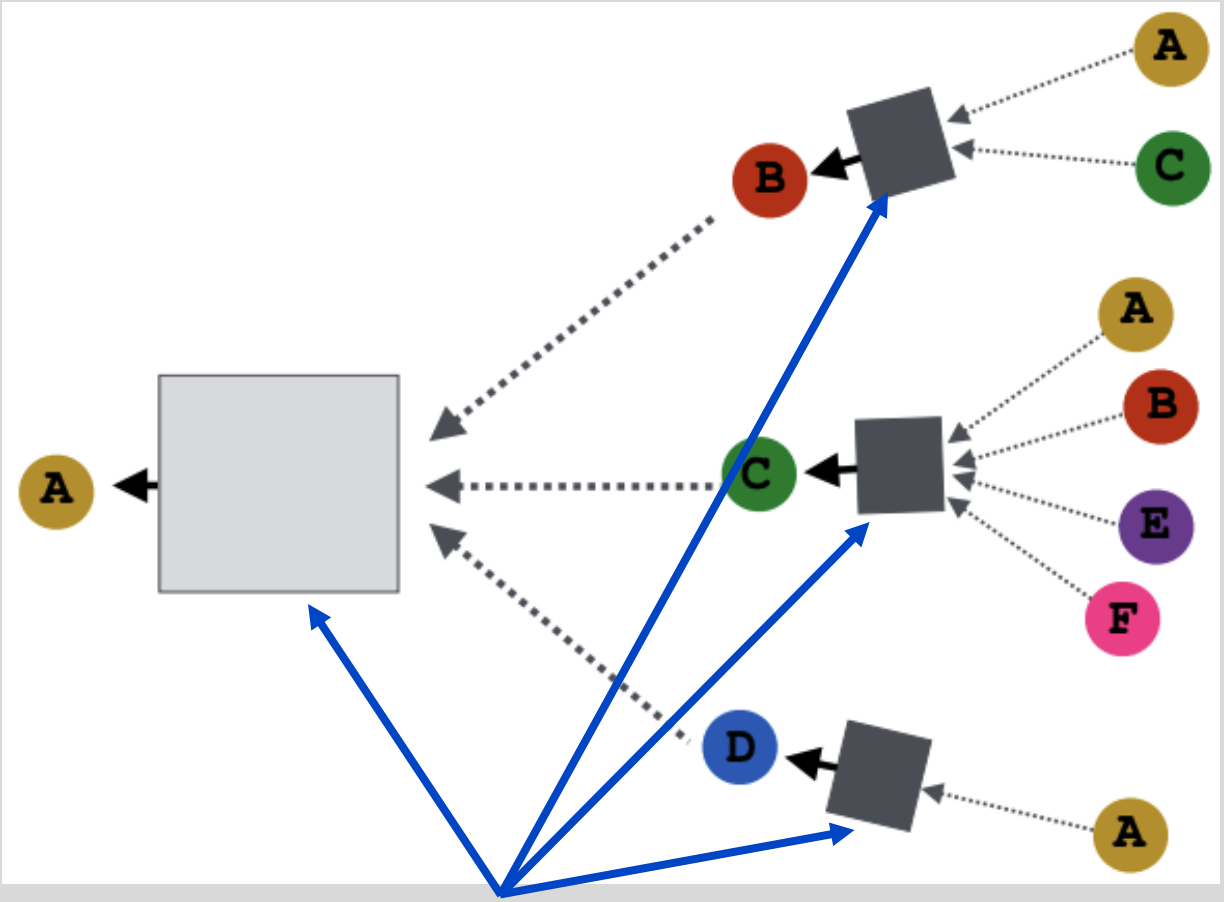


Image credit: Jure Leskovec, http://cs224w.stanford.edu

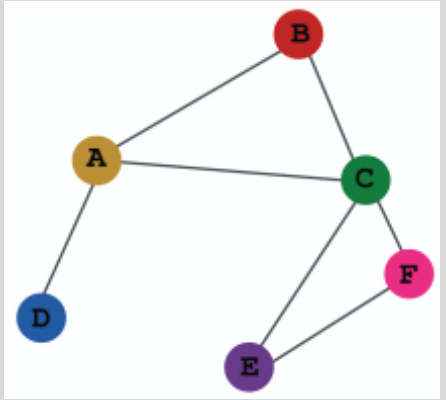


Neural networks

Neighbor node aggregation

- Every neighborhood defines a computation graph

Input graph



Each node has a unique computation graph based on its neighborhood

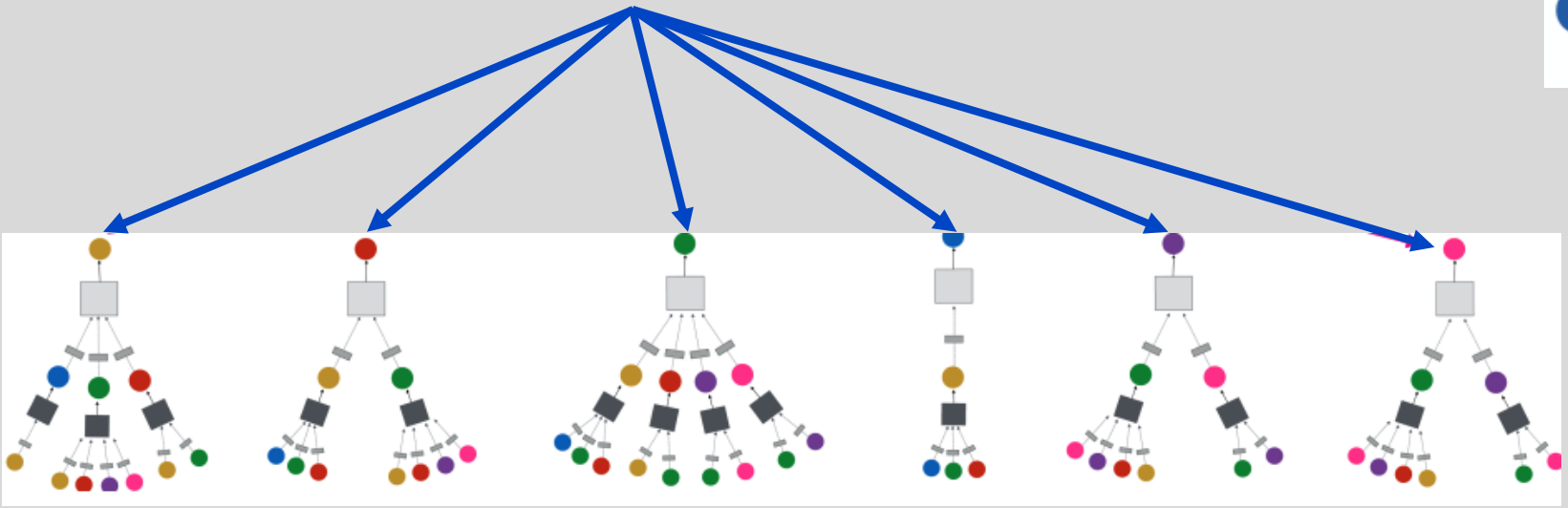
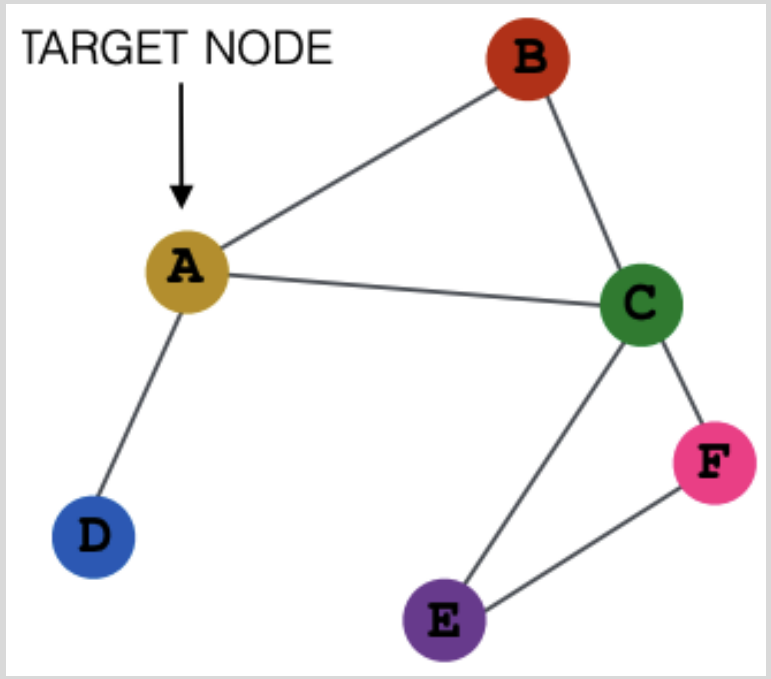


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Neighbor node aggregation and layers

- Models can be of arbitrary depth
  - Nodes have embeddings at each layer
  - Layer-0 embedding of node v is its input vector, $\mathbf{x}_v$
  - Layer-k embedding gets information from nodes that are k hops (edges) away

Input graph



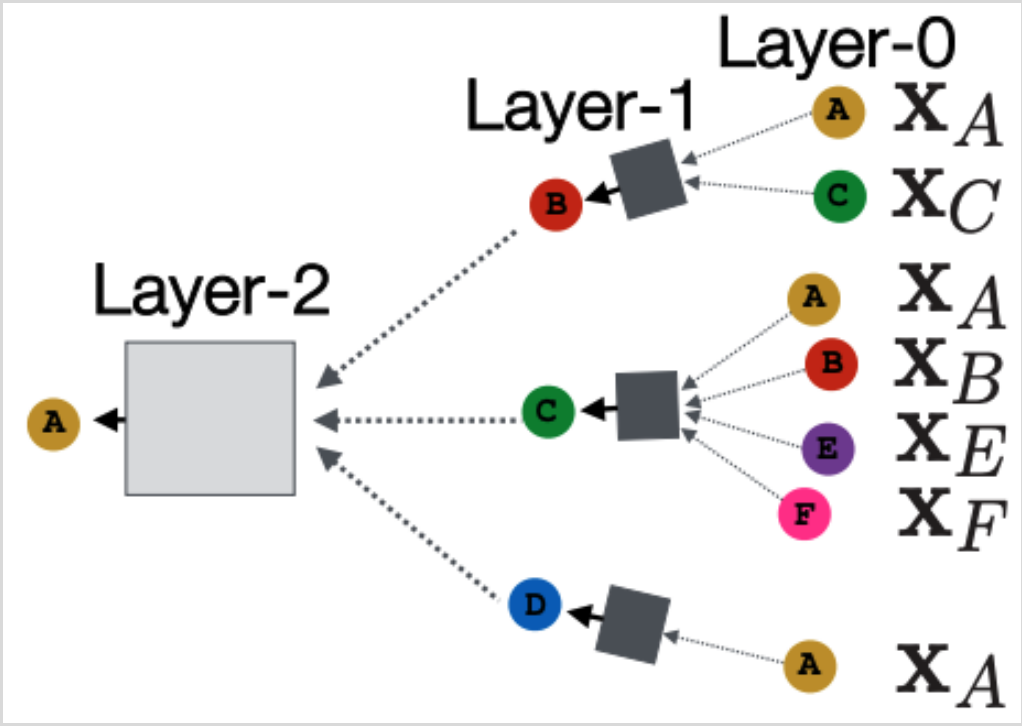


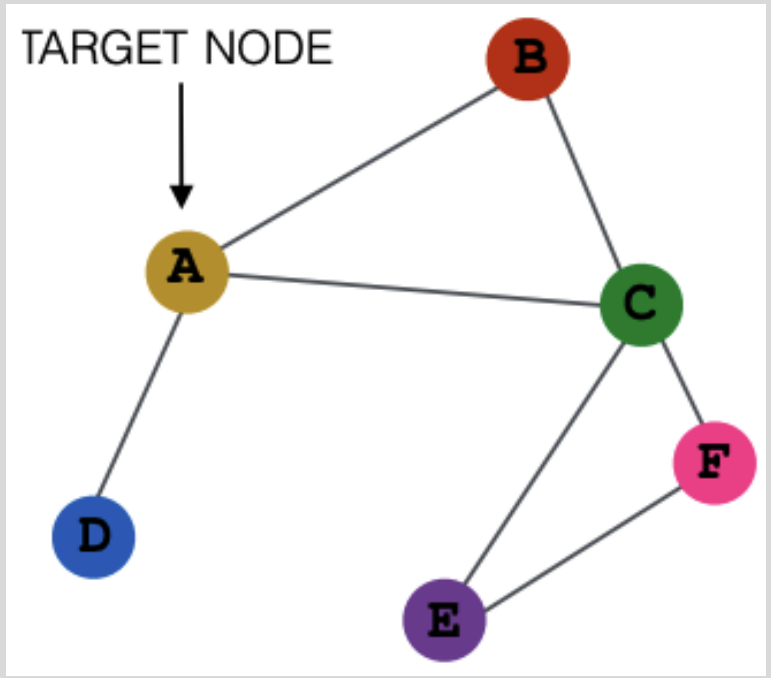
Image credit: Jure Leskovec, http://cs224w.stanford.edu

Neighbor node computation

- One approach: Average the *messages* (feature vectors) from node neighbors and then apply a neural network

(1) message averaging

Input graph



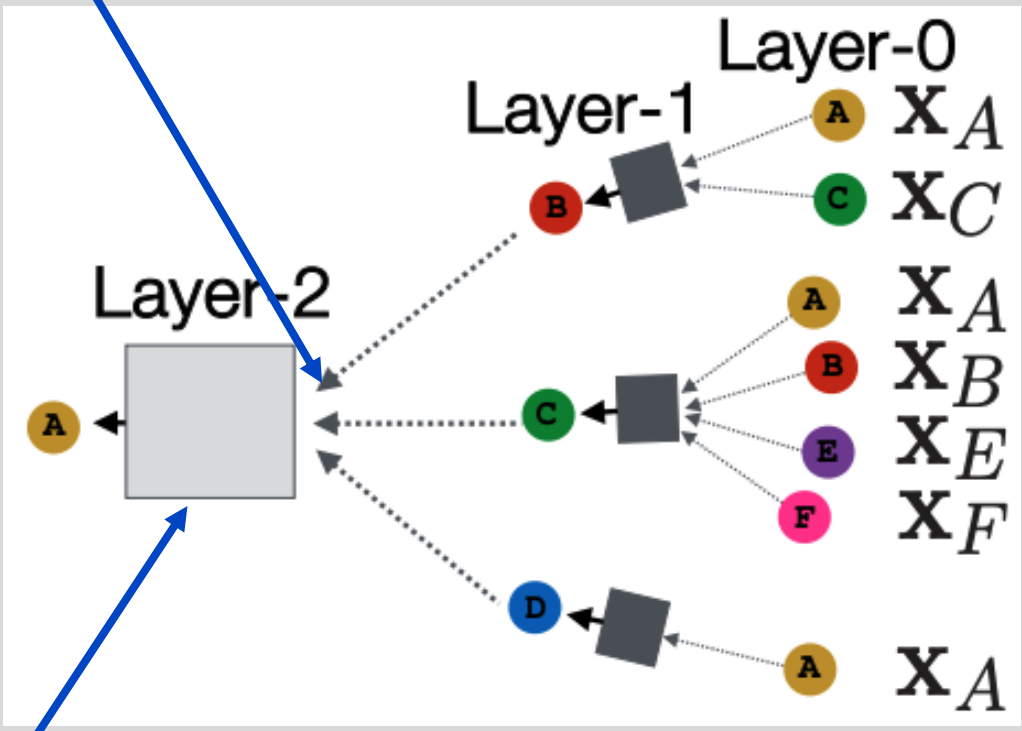


Image credit: Jure Leskovec, http://cs224w.stanford.edu

(2) neural network

Neighbor node computation

- **Self loops**: To reduce loss of self information, self loops are added so that a node's own feature vector is used in it's update

Input graph

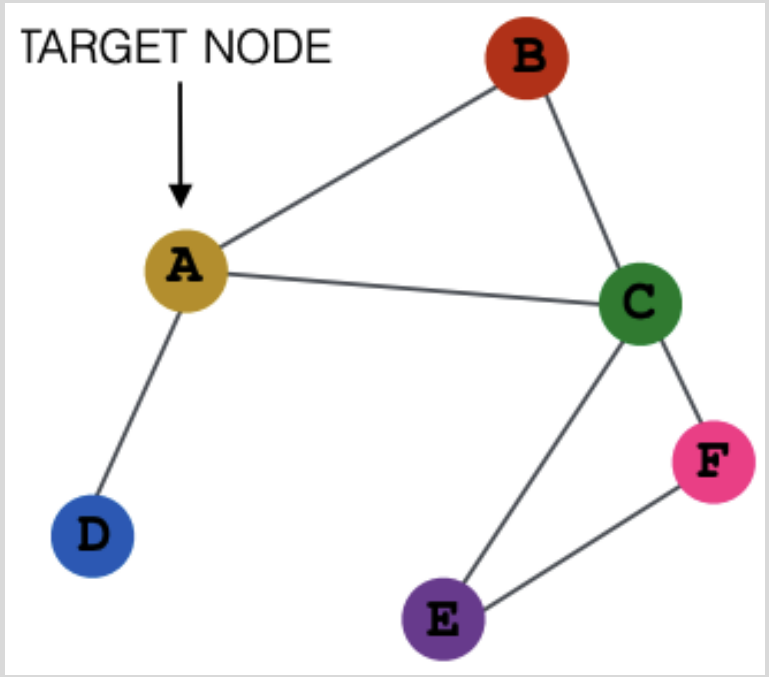
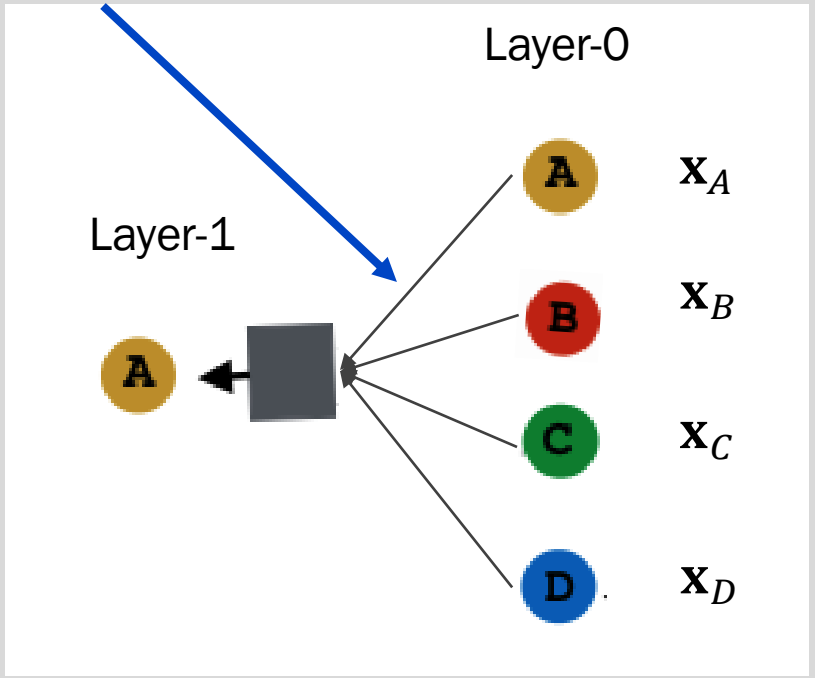


Image credit: Jure Leskovec, http://cs224w.stanford.edu

(1) self loop



The math of graph convolution networks

- Consider a single node v with neighbors in $N(v)$
- Let $\mathbf{h}_v^k$ be the latent representation of v at layer k
- $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the input feature vector for v
- At successive layers, where $k \in [0, \cdots K-1]$, K is the number of layers and σ is a non-linear activation

$$\mathbf{h}_v^{(k+1)} = \sigma\left(W^{(k)} \sum_{u \in N(v) \cup \{v\}} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}\right)$$

Learnable weight matrix

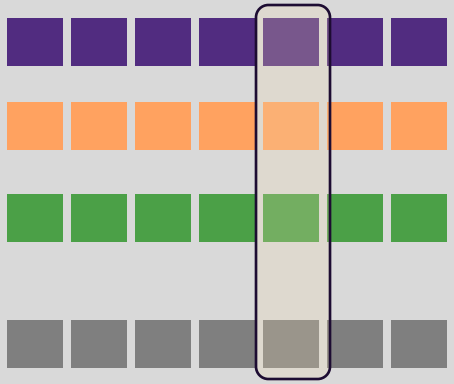
- $\mathbf{h}_v^{(K)} = \mathbf{z}_v$

Final node embedding

We can input the final embeddings into any loss function and learn the weight parameters using back propagation and SGD

Element-wise vector summation

Previous slide, we had $\sum_{u \in N(v) \cup \{v\}} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}$



$\mathbf{h}_1^{(k)}$ embedding

$\mathbf{h}_2^{(k)}$ embedding

$\mathbf{h}_3^{(k)}$ embedding

$\sum_j \mathbf{h}_j^{(k)}$ embedding

The math of graph convolution networks in matrix form

- It will be more efficient if we can put the update equations in matrix form and leverage sparse matrix operations
- Let $\mathbf{H}^{(k)} = \begin{bmatrix} h_1^k \\ \vdots \\ h_{[V]}^k \end{bmatrix}$ be the matrix of latent node representations $\forall v \in G$
- Then letting $\mathbf{A}_{v,:}$ be the v^{th} row of the adjacency matrix (with $a_{v,v} = 1$):

$$\sum_{u \in N(v) \cup \{v\}} \mathbf{h}_u^{(k)} = \mathbf{A}_{v,:}\mathbf{H}^{(k)}$$

- Now, let $\mathbf{D}$ be a diagonal matrix with diagonal entries $\mathbf{D}_{v,v} = deg(v) = |N(v)|$
- Then $\mathbf{D}^{-1}$ is also diagonal with entries $\mathbf{D}_{v,v}^{-1} = {}^{1}/_{|N(v)|}$
- Finally, we can write the update function across all nodes as

$$\sum_{u \in N(v) \cup \{v\}} \frac{\mathbf{h}_u^{(k)}}{|N(v)|} \Rightarrow \mathbf{H}^{(k+1)} = \mathbf{D}^{-1}\mathbf{A}\mathbf{H}^{(k)}$$

The math of graph convolution networks in matrix form

- From the previous result, we can rewrite the update function over all nodes as:

$$\mathbf{H}^{(k+1)} = \sigma\left(\tilde{\mathbf{A}}\mathbf{H}^{(k)}\mathbf{W}^{(k)^T}\right)$$

where $\tilde{\mathbf{A}} = \mathbf{D}^{-1}\mathbf{A}$

normalization

neighborhood aggregation

- $\tilde{\mathbf{A}}$ is sparse which allows computationally efficient sparse matrix multiplication
- Not all GNNs can be expressed in matrix form (depends on the neighborhood aggregation function)

Node embeddings are permutation invariant

The GCN that computes a given node's embedding is independent of the ordering of the node neighbors due to NNet weight sharing and neighbor averaging

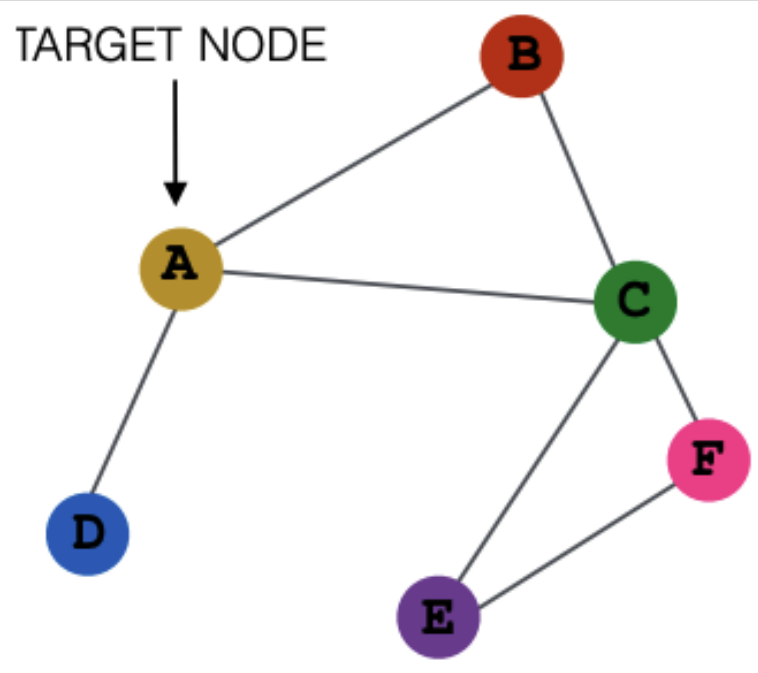
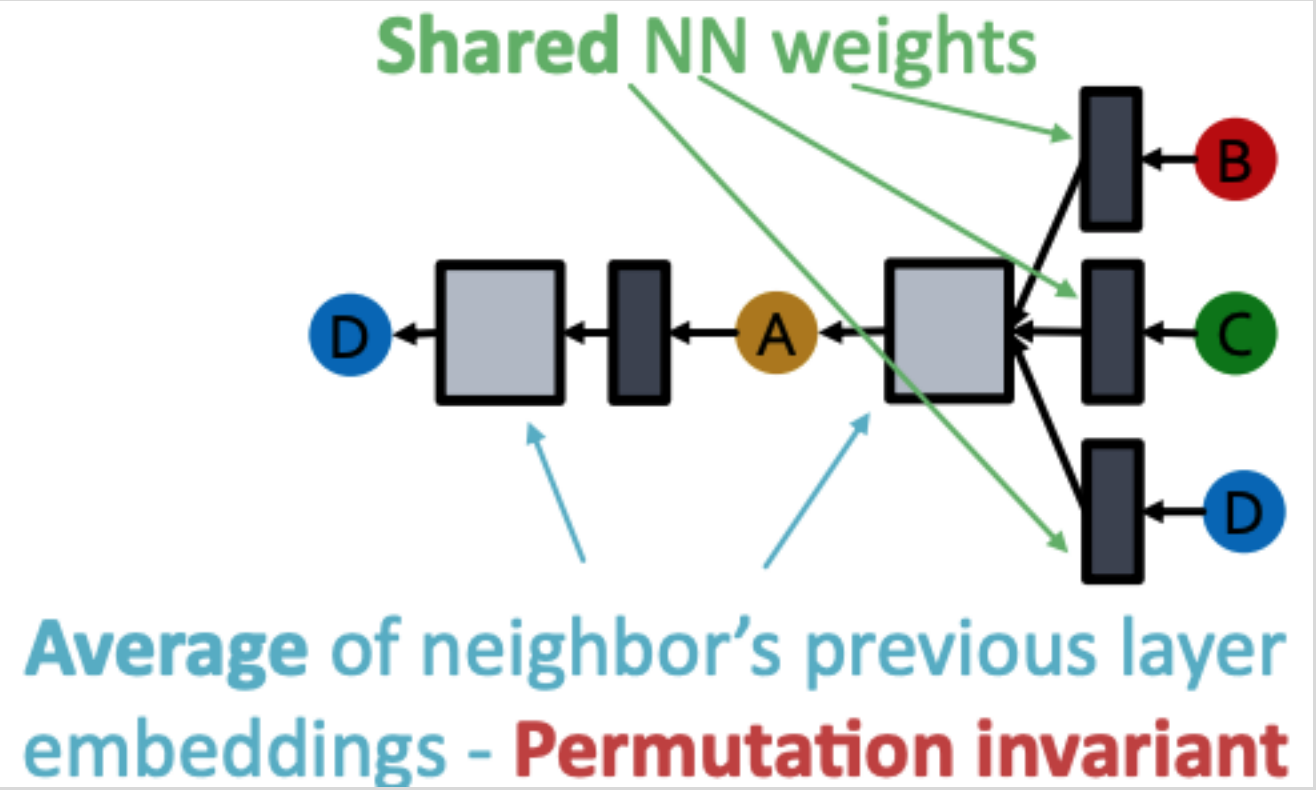
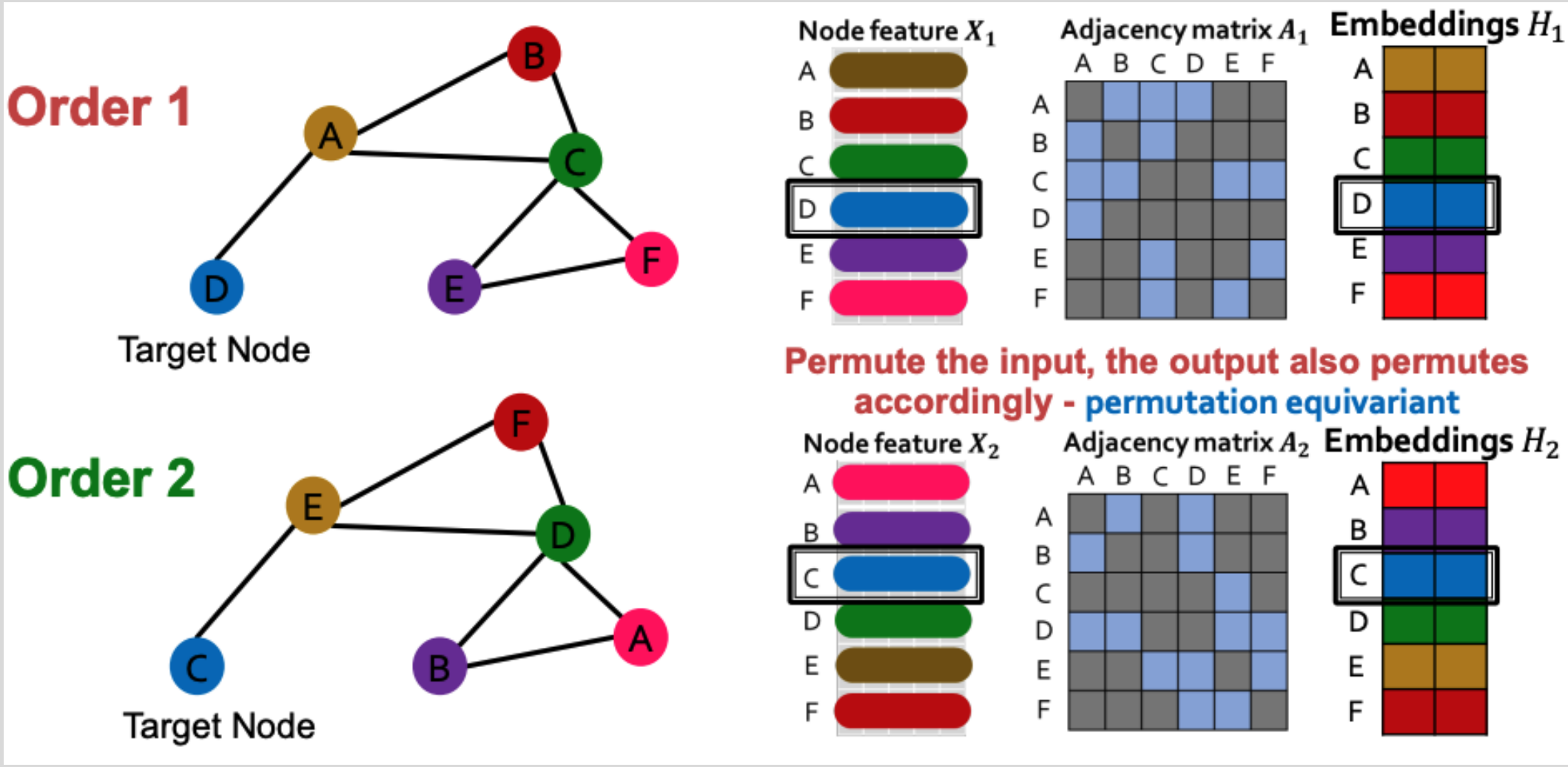


Image credit: Jure Leskovec, http://cs224w.stanford.edu

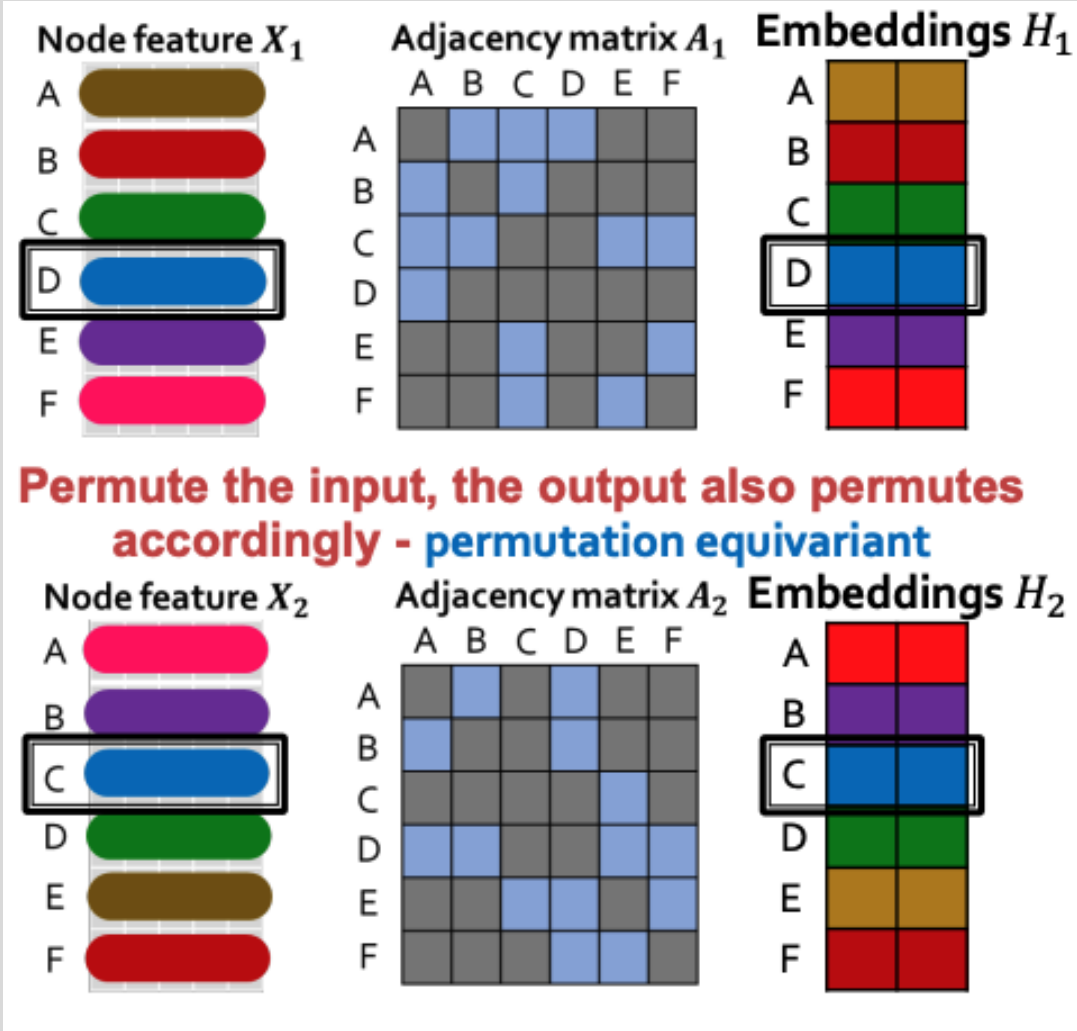


Embedding matrix of all nodes is permutation equivariant



Embedding matrix of all nodes is permutation equivariant

1. The input node features and output embeddings have the same row indices.
2. We know the embedding computation of a given node with a GCN is permutation invariant.
3. So, permuting moves a given node to a new row in the feature matrix and the adjacency matrix, but also moves the output embedding to the same new row in the embedding matrix. This is permutation equivariance.



Training a GNN

- Supervised setting: Minimize a loss function, $\mathcal{L}$, with respect to model parameters

$$\min_{\Theta} \mathcal{L}\left(y, f_{\Theta}(\boldsymbol{x})\right)$$

  - y is a node, edge, or graph target value
  - $f_{\Theta}(\boldsymbol{x}) = DEC\left[ENC\left((\boldsymbol{x})\right)\right]$ is a composite function that includes the DEC (typically another NNet) which outputs the estimate, $\hat{y}$, of y given the ENC output which is a node, edge, or graph embedding from a GNN
  - $\mathcal{L}$ is any appropriate loss function, e.g., L2 if y is a real number, or cross entropy if y is categorical
- Unsupervised setting
  - No labels available
  - Use the graph structure as self-supervision (e.g., link prediction)

Unsupervised GNN training

- One approach is to assume *similar* nodes should have similar embeddings using cross entropy loss:

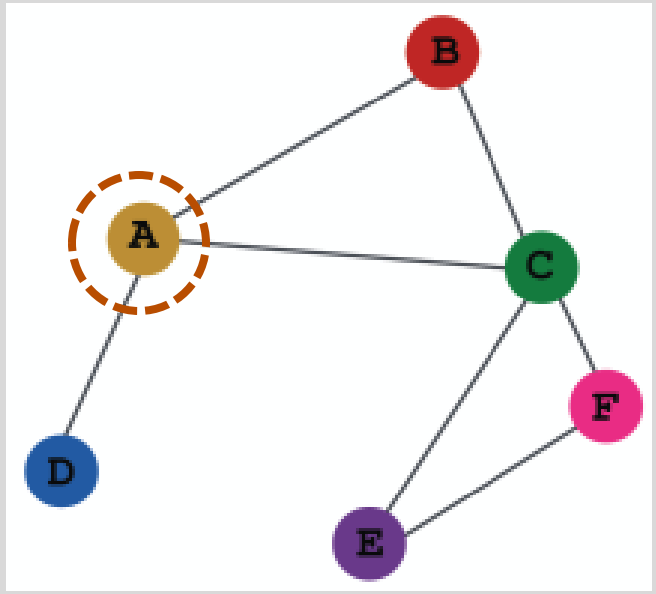
$$\min_{\Theta} \mathcal{L} = -\sum_{u,v} \sum_{i=1}^{C} \left(y_i log(\mathbf{z}_u \cdot \mathbf{z}_v) \right)$$

  where y_i is the actual class label based on a similarity function, S_i. That is, $y_i = 1$ if $S_i(u, v) > \gamma$ for some threshold γ.
- Can apply any node similarity function, e.g., random walks

Supervised GNN training

- As with standard deep-learning, directly train the model for a supervised task such as node classification

Drug-drug interaction network



Is the drug safe or toxic?

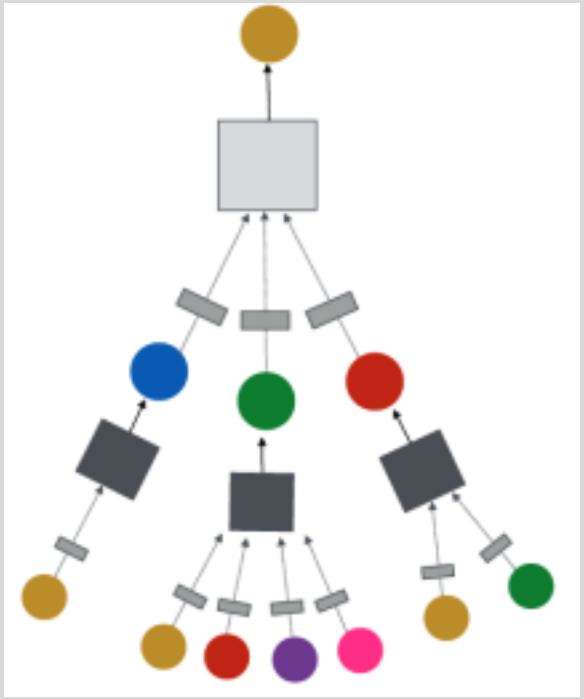


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Supervised GNN training

- Directly train the model for a supervised task such as node (or graph or edge) classification
- Apply cross-entropy loss

$$\mathcal{L} = -\sum_{v \in V} y_v \log\left[\sigma\left(DEC(\mathbf{z}_v)\right)\right] + (1 - y_v)\log\left[1 - \sigma\left(DEC(\mathbf{z}_v)\right)\right]$$

class label

encoder output (embedding)

- Loss gradient is computed for composite function $DEC(ENC(v))$ and learning parameters are updated for both
  - DEC can be just the identify function, i.e. just use $\sigma(\mathbf{z}_v)$

Safe or toxic?

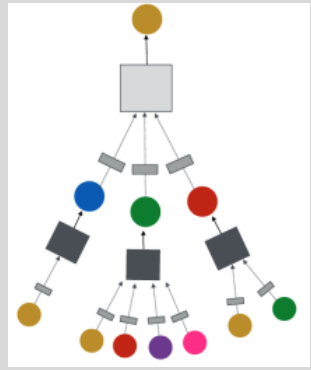
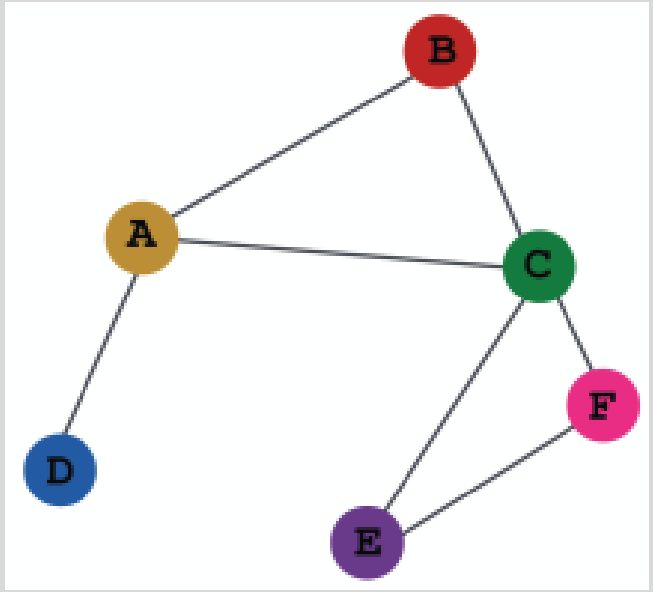
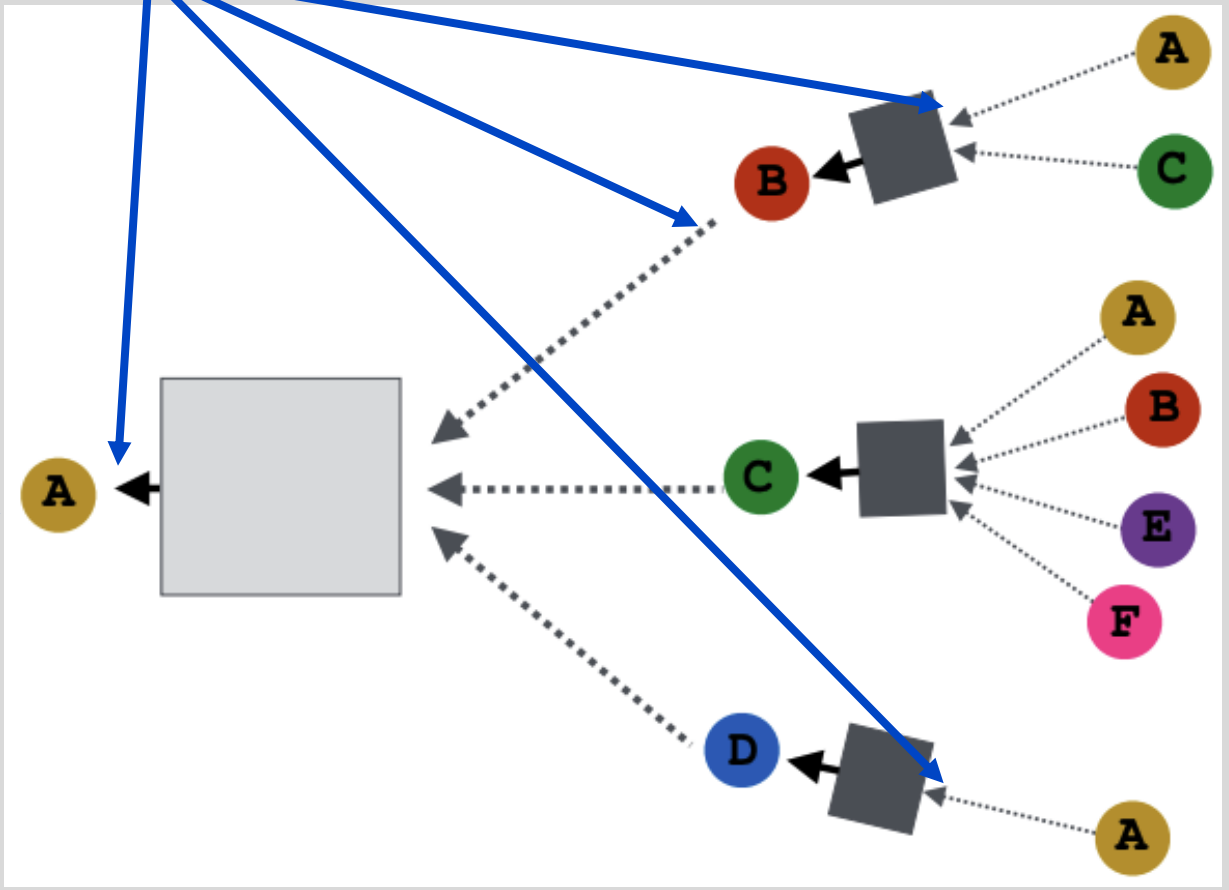


Image credit: Jure Leskovec, http://cs224w.stanford.edu

GNN model design overview

(1) Define the neighborhood aggregation function

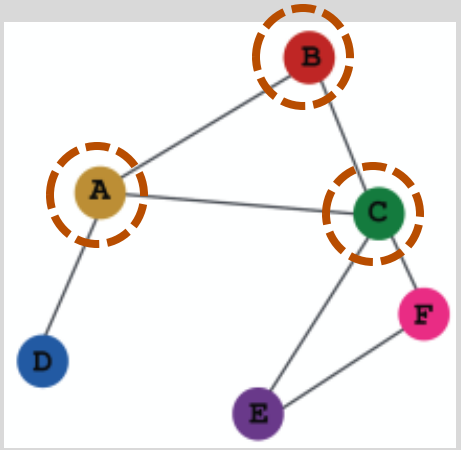




(2) Define a loss function

Image credit: Jure Leskovec, http://cs224w.stanford.edu

GNN model design overview



(3) Select training nodes and train the model with SGD

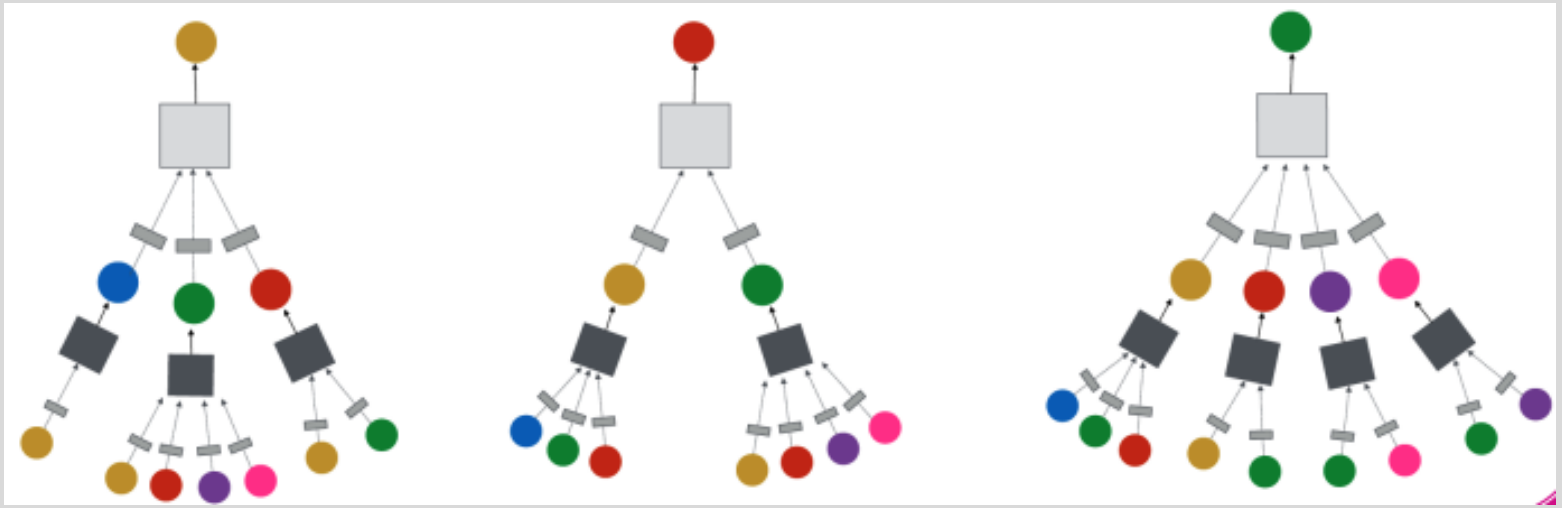
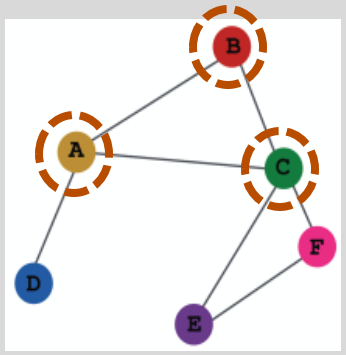


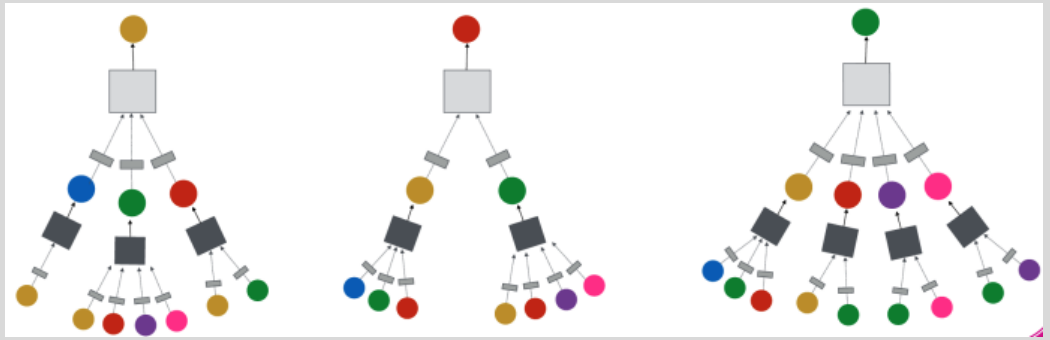
Image credit: Jure Leskovec, http://cs224w.stanford.edu

GCN embedding generation

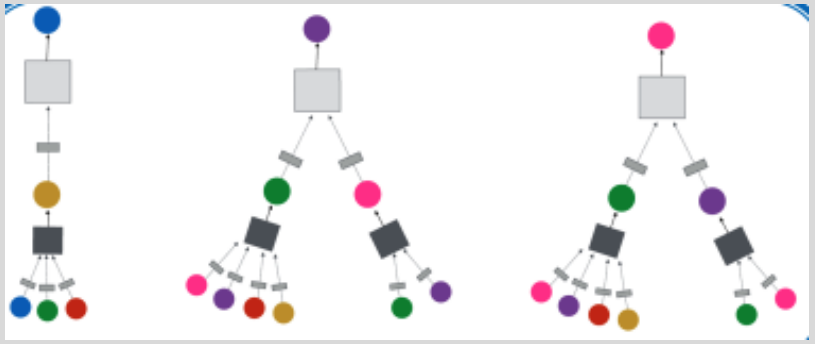
Once the model is trained, we can generate embeddings for **any** nodes



Nodes seen in training



Nodes not seen in training



How is this possible?

GCN inductive capacity for unseen or new nodes

- The same aggregation learning parameters are shared for all nodes (or edges)
- Different parameters at each layer, i.e., $W_k \neq W_j$ for $k \neq j$
- What does this imply in the math?

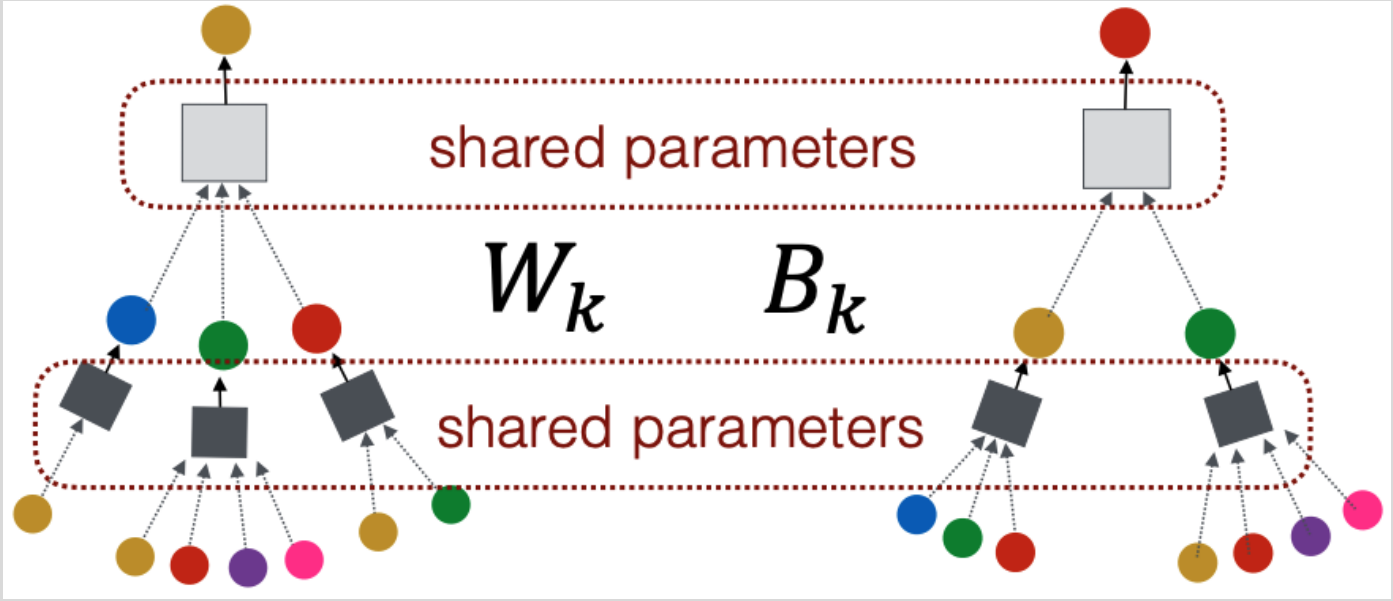


Image credit: Jure Leskovec, http://cs224w.stanford.edu

GCN inductive capacity for unseen or new nodes

Recall the math $\mathbf{H}^{(k+1)} = \sigma\left(\widetilde{\mathbf{A}}\mathbf{H}^{(k)}\mathbf{W}^{(k)^T}\right)$

- $\widetilde{\mathbf{A}} \in \mathbb{R}^{|V|\times|V|}$
- $\mathbf{H}^{(k)} \in \mathbb{R}^{|V|\times \boldsymbol{d}_k}$
- $\mathbf{W}^{(k)^T} \in \mathbb{R}^{\boldsymbol{d}_k \times \boldsymbol{d}_{k+1}}$

where $\boldsymbol{d}_k$ and $\boldsymbol{d}_{k+1}$ are the latent embedding dimensions at layer k and $k+1$, respectively

The operation $\widetilde{\mathbf{A}}\mathbf{H}^{(k)} \in \mathbb{R}^{|V|\times \boldsymbol{d}_k}$ and $\widetilde{\mathbf{A}}\mathbf{H}^{(k)}\mathbf{W}_k^T \in \mathbb{R}^{|V|\times \boldsymbol{d}_{k+1}}$

- The weight matrix is not dependent on the adjacency matrix dimensions
- We can create a new adjacency matrix to add or remove nodes (and edges) and process it with the same trained model to get embeddings

GCN inductive capacity for unseen or new nodes

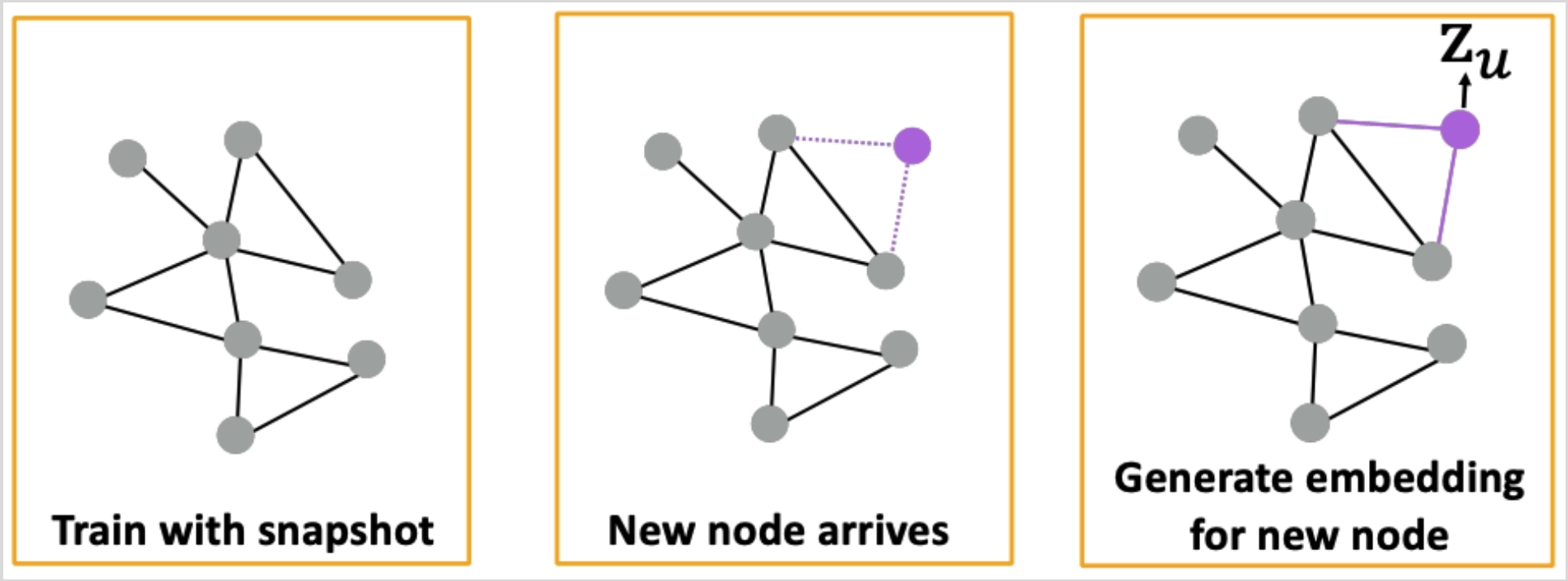


Image credit: Jure Leskovec, http://cs224w.stanford.edu

- Many applications have dynamic graphs (social networks, device networks)
- Can generate new embeddings as needed and pass into downstream inference models
- Assumes new nodes have same features and inter-node similarity properties hold

GCN inductive capacity for new graphs

- Inductive node embeddings can generalize to an entirely unseen graph (it's just a new adjacency matrix)
- Example: train on known drug molecules and generate embeddings new molecules

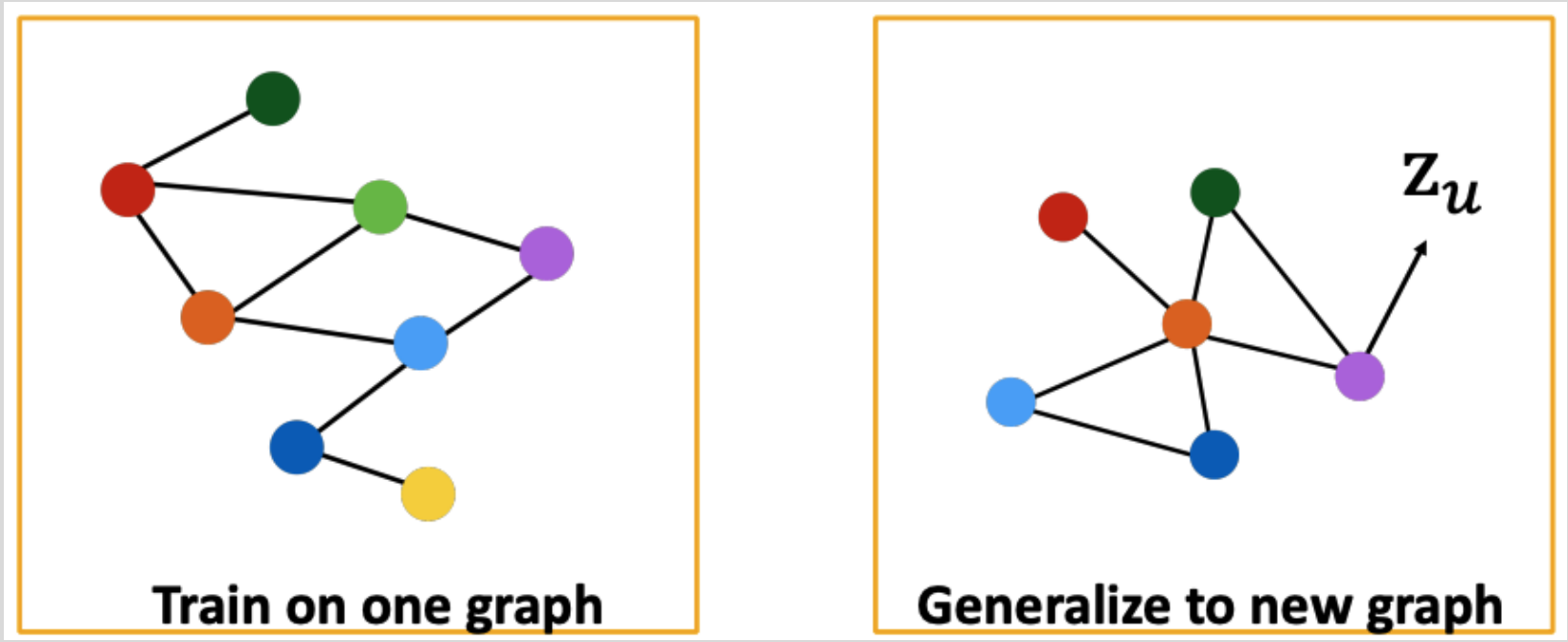


Image credit: Jure Leskovec, http://cs224w.stanford.edu

More on GNNs

Interactive visualizations

https://visual-intelligence-umn.github.io/GNN-101/